МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Санкт–Петербургский государственный университет
аэрокосмического приборостроения»

Кафедра №43 Институт информационных технологий и программирования

ОТЧЁТ ПО ПРАКТИКЕ
ЗАЩИЩЁН С ОЦЕНКОЙ

РУКОВОДИТЕЛЬ

старший преподаватель		А.А. Фоменкова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЁТ ПО ПРАКТИКЕ

вид практики Учебная

тип практики

на тему индивидуального задания Реализация численных методов в среде MATLAB

(в соответствии с вариантом)

выполнен Панченко Андреем

фамилия, имя, отчество обучающегося в творительном падеже

по направлению подготовки 02.03.03 Математическое обеспечение и

код наименование направления

администрирование информационных систем

наименование направления

направленности

код наименование направленности

наименование направленности

Обучающийся группы № 4236		А. Панченко
номер	подпись, дата	инициалы, фамилия

Санкт-Петербург 2023

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

на прохождение учебной ознакомительной практики обучающегося направления подготовки/ специальности математическое обеспечение и администрирование информационных систем

1. Фамилия, имя, отчество обучающегося: Панченко Андрей

2. Группа: 4236

3. Тема индивидуального задания:
Реализация численных методов в среде MATLAB (в соответствии с вариантом)

4. Исходные данные:
Учебно-вычислительная практика: методические указания для студентов очной форм обучения / С.-Петерб. гос. ун-т аэрокосм. приборостроения ; сост.: А. А. Ключарев, М. В. Фаттахова. - СПб. : Изд-во ГУАП, 2014.

5. Содержание отчетной документации:

5.1. индивидуальное задание;

5.2. отчёт, включающий в себя:

– титульный лист;

– материалы о выполнении индивидуального задания (содержание определяется кафедрой);

– выводы по результатам практики;

– список использованных источников.

5.3. отзыв руководителя от профильной организации (при прохождении практики в профильной организации).

6. Срок представления отчета на кафедру: «01» июня 2023 г.

Руководитель практики

Ст. преподаватель		А. А. Фоменкова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

СОГЛАСОВАНО

Руководитель практики от профильной организации

должность	подпись, дата	инициалы, фамилия

Задание принял к исполнению:

Обучающийся

«09» февраля 2023 г.		Панченко А.
дата	подпись	инициалы, фамили

Санкт-Петербург 2023

Содержание

Введение 4
1. Задание по варианту 5
2. Решение нелинейного уравнения методом Ньютона (касательных). 6
2.1 Математическая постановка задачи метода Ньютона (касательных) 6
2.2 Описание метода численной реализации задания 6
2.3 Описание ручного счета тестового примера предложенным методом 7
2.4 Разработка алгоритма решения задачи 10
2.5 Блок-схема 12
2.6 Результат работы программы 13
3. Интерполирование функции по формулам Ньютона 14
3.1 Математическая постановка задачи интерполяции функции методом Ньютона 14
3.2 Описание метода численной реализации задания 15
3.3 Описание ручного счета тестового примера предложенным методом 16
3.4 Разработка алгоритма решения задачи 18
3.5 Блок-схема 19
3.6 Результат работы программы 20
4. Численное интегрирование методом Симпсона 21
4.1 Математическая постановка интегрирование методом Симпсона 21
4.2 Описание метода численной реализации задания 21
4.3 Описание ручного счета тестового примера предложенным методом 22
4.4 Разработка алгоритма решения задачи 24
4.5 Блок-схема 25
4.6 Результат работы программы 26
5. Работа программы, виды неправильного и правильного ввода пользователем 27
Заключение 41
Список использованных источников 42
Приложение А Листинги программ 43
Приложение А.1 Файл «eto_1_razdel» 43
Приложение А.2 Файл «eto_2_razdel» 46
Приложение А.3 Файл «eto_3_razdel» 51
Приложение В Полный код 54
Приложение С Основные функции решения программы 55
Приложение С.1 Файл «nyton» 55
Приложение С.2 Файл «evaluate_polynomial» 56
Приложение С.3 Файл «sympson_kavo» 57

Введение

Учебная ознакомительная практика Санкт-Петербургского государственного университета аэрокосмического приборостроения кафедры №43 “ Институт информационных технологий и программирования”. Выполнение индивидуального задания учебной практики осуществляется в среде программирования MATLAB2022a на языке программирования MATLAB.

В данном отчете рассмотрены результаты выполнения задания учебной практики, представлен анализ написанного алгоритма, а также прикреплены скриншоты результатов работы программной функции. Приложение к отчету содержит код реализованного алгоритма. Работа выполнена на основе программы весеннего семестра с целью закрепления приобретенных знаний.

Задание учебной практики заключается в том, чтобы разработать приложение с текстовым или графическим (по желанию студента) интерфейсом пользователя, реализующее алгоритмы заданных математических методов в виде функций на языке программирования MATLAB в среде MATLAB, FreeMat, Octave или SciLab. Для недопустимых значений входных данных, при которых невозможно провести вычисления, должно отображаться сообщение об ошибке. Также необходимо провести сравнение разработанных функций со встроенными функциями используемого математического пакета (MatLab, FreeMat, Octave), решающими те же задачи.

1. Задание по варианту

Варианты индивидуальных заданий представлены на рисунке 1.

№ п/п	ФИО	Раздел 1			Раздел 2			Раздел 3			ОТЧЁТ	Премиальные баллы	Сумма	Итоговая оценка
		1 этап	2 этап	3 этап	1 этап	2 этап	3 этап	1 этап	2 этап	3 этап				
	Максимальные баллы	8	3	6	8	3	6	8	3	6	49	10	110	
	СРОК СДАЧИ	02.мар	02.мар	09.мар	23.мар	30.мар	06.апр	20.апр	27.апр	04.май	25.май			
1	Атюцкий Н. М.													
2	Батин В. С.													
3	Богданов Р. Р.													
4	Борщ Н. С.													
5	Гинин Н. В.													
6	Гончарук Е. О.													
7	Долина Н. М.													
8	Жуковский С. А.													
9	Завгородний А. Р.													
10	Зайцев А. В.													
11	Кельбина К. А.													
12	Колесникова Л. Э.													
13	Литвинова А. М.													
14	Лукина А. А.													
15	Мачульский Д. Р.													
16	Мвале Л. .													
17	Николаев А. А.													
18	Панин А. К.													
19	Панченко А. .													
20	Полпудников В. О.													
21	Семенюк А. М.													
22	Супруненко Г. А.													
23	Щукин А. А.													
24	Юрьев Р. А.													

Рисунок 1 – Варианты индивидуальных заданий

2. Решение нелинейного уравнения методом Ньютона (касательных).

2.1 Математическая постановка задачи метода Ньютона (касательных)

Дано нелинейное уравнение вида f(x) = 0, где f(x) является дифференцируемой функцией.

Требуется решить данное уравнение методом Ньютона (касательных), то есть найти корень уравнения x* такой, что f(x*) = 0.[1]

Метод Ньютона (касательных) заключается в следующем: на каждой итерации находится касательная к графику функции f(x) в точке x_i , затем пересечение этой касательной с осью абсцисс даёт новую оценку x_{i+1} корня уравнения. Таким образом, последовательность оценок корня x_i строится по формуле:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}, \quad (2.1)$$

где $f'(x_i)$ - производная функции f(x) в точке x_i.

Алгоритм заканчивает работу, когда достигнута заданная точность или выполнен критерий останова, например, если достигнуто максимальное количество итераций.[1]

2.2 Описание метода численной реализации

Метод Ньютона (касательных) является итерационным методом решения нелинейных уравнений, который основан на использовании касательных к графику функции. Ниже приведен общий алгоритм численной реализации метода Ньютона (касательных):

Задать начальное приближение x0, точность eps и максимальное число итераций max_iter.

На каждой итерации i вычисляются значения функции $f(x_i)$ и её производной $f'(x_i)$ в точке x_i.

Вычислить новое приближение к корню по формуле:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (2.2)$$

Если $|x_{i+1} - x_i| <$eps или достигнуто максимальное число итераций, то алгоритм завершается и возвращается приближение к корню x_{i+1}. Иначе алгоритм продолжает работу, переходя к следующей итерации.[2]

После завершения работы алгоритма возвращается найденное приближение к корню $x_{(i+1)}$.

Важно заметить, что при использовании метода Ньютона (касательных) могут возникать проблемы, если производная $f'(x_i)$ близка к нулю в точке x_i или если алгоритм расходится. Для предотвращения таких проблем может быть применена модификация метода Ньютона, например, метод с использованием золотого сечения или метод с использованием релаксации.

2.3 Описание ручного счета тестового примера предложенным методом

С помощью графического метода найти промежуток [a; b], на котором находится действительный корень x* уравнения $x^3+4x-3=0$. Пользуясь методом Ньютона, получить приближенное значение корня с точностью до 0,001

Решение:

На первом шаге следует отделить корень графически. Это можно сделать путём построения графика $f(x)=x^3+4x-3$, но такой подход обладает рядом недостатков. Во-первых, не факт, что график прост , а программное обеспечение – оно далеко не всегда под рукой. И, во-вторых (следствие из 1-го), с немалой вероятностью получится даже не схематичный чертёж, а грубый рисунок, что, разумеется, не есть хорошо.

Представим уравнение $x^3+4x-3=0$ в виде $x^3=3-4x$,построим графики $g(x)=x^3$,$h(x)=3-4x$ и отметим на чертеже корень x* («иксовую» координату точки пересечения графиков):

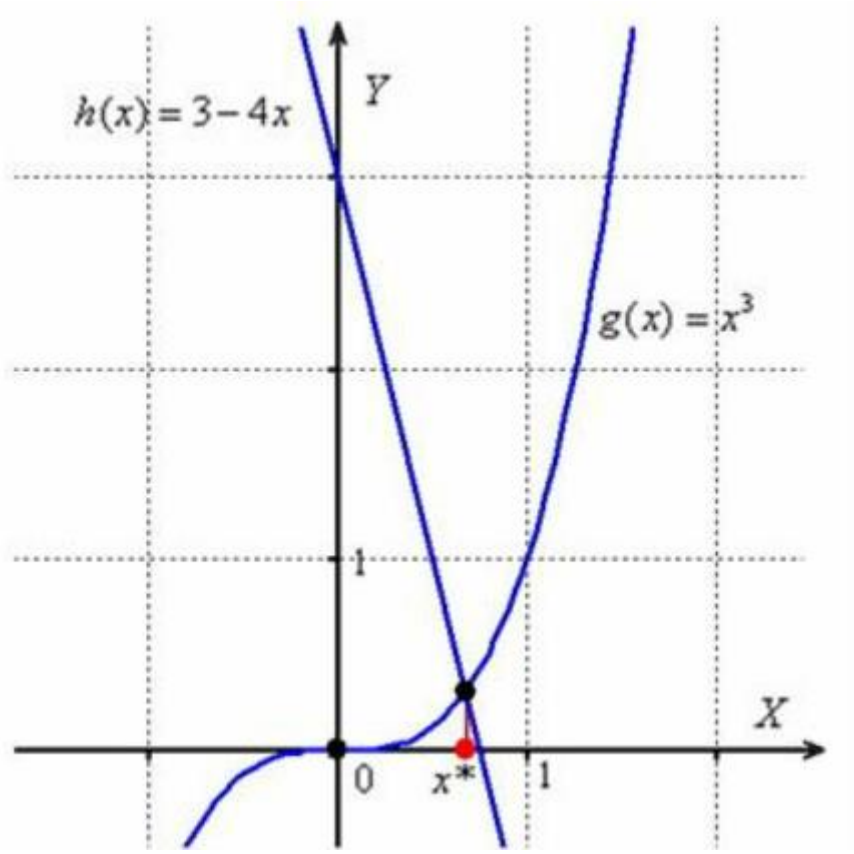


Рисунок 2 – Пересечение графиков с корнем

Очевидное преимущество этого способа состоит в том, что графики данных функций строятся от руки значительно точнее и намного быстрее. Заметим, что прямая пересекла кубическую параболу в единственной точке, а значит, предложенное уравнение имеет только один действительный корень.[3]

Итак, x* принадлежит отрезку [0;1] и примерно равен 0,65-0,7.

На втором шаге нужно выбрать начальное приближение x_0 корня. Обычно это один из концов отрезка. Начальное приближение должно удовлетворять следующему условию:

$$f(x_0) * f''(x_0) > 0 \qquad (2.3)$$

Найдём первую и вторую производные функции $f(x)=x^3+4x-3$:

$f'(x)=(x^3+4x-3)'=3x^2+4$

$f''(x) =(3x^2+4)'=6x$

и проверим левый конец отрезка:

$f(0) =0*3+4*0-3= -3$, $f''(0) =6*0=0$

$f(0) *f''(0) = -3*0 = 0$

Таким образом, ноль «не подошёл».

Проверяем правый конец отрезка:

f (1) =13+4*1-3=2, f'' (1) = 6*1 = 6

f (1)*f'' (1) = 2*6 = 12>0 – всё хорошо! В качестве начального приближения выбираем x_0=1.

На третьем шаге нас ожидает дорога к корню. Каждое последующее приближение корня x_{n+1} рассчитывается на основании предшествующих данных с помощью следующей рекуррентной формулы:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \qquad (2.4)$$

Процесс завершается при выполнении условия $\left|\frac{f(x_n)}{f'(x_n)}\right| < \varepsilon$, где ε – заранее заданная точность вычислений. В результате за приближённое значение корня принимается «энное» приближение: $x^* \approx x_n$.

Расчёты:

$f(x_0) = f(1) = 2$

$f'(x_0) = f'(1) = 3*12+4 = 7$

$\frac{f(x_0)}{f'(x_0)} = \frac{2}{7} \approx 0{,}28571$ (округление обычно проводят до 5-6 знаков после запятой)

Поскольку полученное значение больше ε=0,001, то переходим к 1-му приближению корня:

$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} \approx 1 - 0{,}28571 = 0{,}71429$

Вычисляем:

$f(x_1) \approx f(0{,}71429) = (0{,}71429)^3 + 4*0{,}71429 - 3 \approx 0{,}22157$

$f'(x_1) \approx f'(0{,}71429) = 3*(0{,}71429)^2 + 4 \approx 5{,}53061$

$\frac{f(x_1)}{f'(x_1)} \approx \frac{0{,}22157}{5{,}53061} \approx 0{,}04006 > \varepsilon$, поэтому возникает потребность перейти ко 2-му приближению:

$x_2 = x_1 \frac{f(x_1)}{f'(x_1)} \approx 0{,}71249 - 0{,}04006 = 0{,}67422$

Заходим на следующий круг:

$f(x_2) \approx f(0{,}67422) = (0{,}67422)^3 + 4*0{,}67422 - 3 \approx 0{,}00338$

$f'(x_2) \approx f'(0{,}67422) = 3*(0{,}67422)^2 + 4 \approx 5{,}36373$

$\frac{f(x_2)}{f'(x_2)} \approx \frac{0{,}00338}{5{,}36373} \approx 0{,}00063 < \varepsilon$, таким образом, итерации закончены, и в качестве приближённого значения корня следует взять 2-е приближение, которое в соответствии с заданной точностью нужно округлить до одной тысячной:

$x^* \approx x_2 \approx 0{,}67422 \approx 0{,}674$

На практике результаты вычислений удобно заносить в таблицу, при этом, чтобы несколько сократить запись, дробь часто обозначают через $h_n = \frac{f(x_n)}{f'(x_n)}$:

n	x_n	$f(x_n)$	$f'(x_n)$	$h_n = \frac{f(x_n)}{f'(x_n)}$
0	1	2	7	0,28571
1	0,71429	0,22157	5,53061	0,04006
2	0,67422	0,00338	5,36373	0,00063

Таблица 1 – Значения

Сами же вычисления по возможности лучше провести в Экселе – это намного удобнее и быстрее.

Ответ: с точностью до 0,001.

2.4 Разработка алгоритма решения задачи

Алгоритм нахождения корней приближенными методами можно разбить на два этапа. На первом изучается расположение корней и проводится их разделение. Находится область [a, b], в которой существует корень уравнения или начальное приближение к корню x_0. Простейший способ решения этой задачи является исследование графика функции f(x). В общем же случае для её решения необходимо привлекать все средства математического анализа.

Существование на найденном отрезке [a, b], по крайней мере, одного корня уравнения следует из условия Больцано:

$$f(a)*f(b) < 0 \qquad (2.5)$$

При этом подразумевается, что функция f(x) непрерывна на данном отрезке. Однако данное условие не отвечает на вопрос о количестве корней уравнения на заданном отрезке [a,b]. Если же требование непрерывности функции дополнить ещё требованием её монотонности, а это следует из знака постоянства первой производной, то можно утверждать о существовании единственного корня на заданном отрезке.[1]

При локализации корней важно так же знание основных свойств данного типа уравнения. К примеру, напомним, некоторые свойства алгебраических уравнений где вещественные коэффициенты:

а) Уравнение степени n имеет n корней, среди которых могут быть как вещественные, так и комплексные. Комплексные корни образуют комплексно-сопряженные пары и, следовательно, уравнение имеет четное число таких корней. При нечетном значении n имеется, по меньшей мере, один вещественный корень.

б) Число положительных вещественных корней меньше или равно числа переменных знаков в последовательности коэффициентов. Замена x на -x позволяет таким же способом оценить число отрицательных корней.[2]

На втором этапе решения уравнения, используя полученное начальное приближение, строится итерационный процесс, позволяющий уточнять значение корня с некоторой, наперед заданной точностью. Итерационный процесс состоит в последовательном уточнении начального приближения. Каждый такой шаг называется итерацией. В результате процесса итерации находится последовательность приближенных значений корней уравнения. Если эта последовательность с ростом n приближается к истинному значению корня x, то итерационный процесс сходится. Говорят, что итерационный процесс сходится, по меньшей мере, с порядком m, если выполнено условие:

где C>0 некоторая константа. Если m=1, то говорят о сходимости первого порядка; m=2 - о квадратичной, m=3 - о кубической сходимостях.

Итерационные циклы заканчиваются, если при заданной допустимой погрешности выполняются критерии по абсолютным или относительным отклонениям, или малости невязки.

2.5 Блок-схема

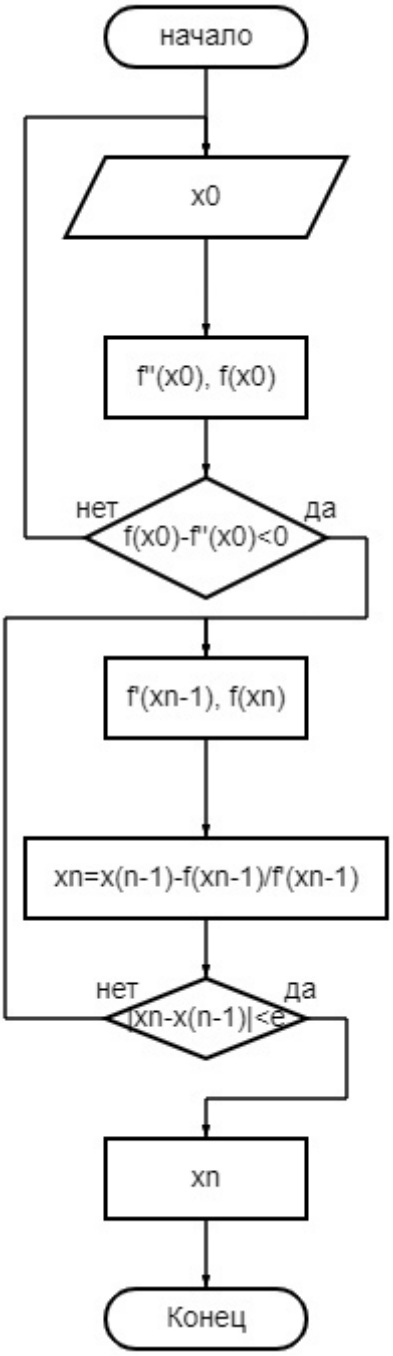


Рисунок 3 – Блок схема решения функции методом Ньютона

2.6 Результат работы программы

```
Command Window
Введите функцию: sin(x)
Введите левое значение оси x: 0
Введите правое значение оси x: 10
Введите начальное приближенное значение x0, используя точку для разделения дробной части: 5
Введите точность вычисления корня e, используя точку для разделения дробной части: 0.001
Уравнение решено за 5 итераций
Корень уравнения: 9.425451
Корень уравнения по встроенной функции fzero: 9.424778
fx >>
```

Рисунок 4 – Вводимые данные и вывод

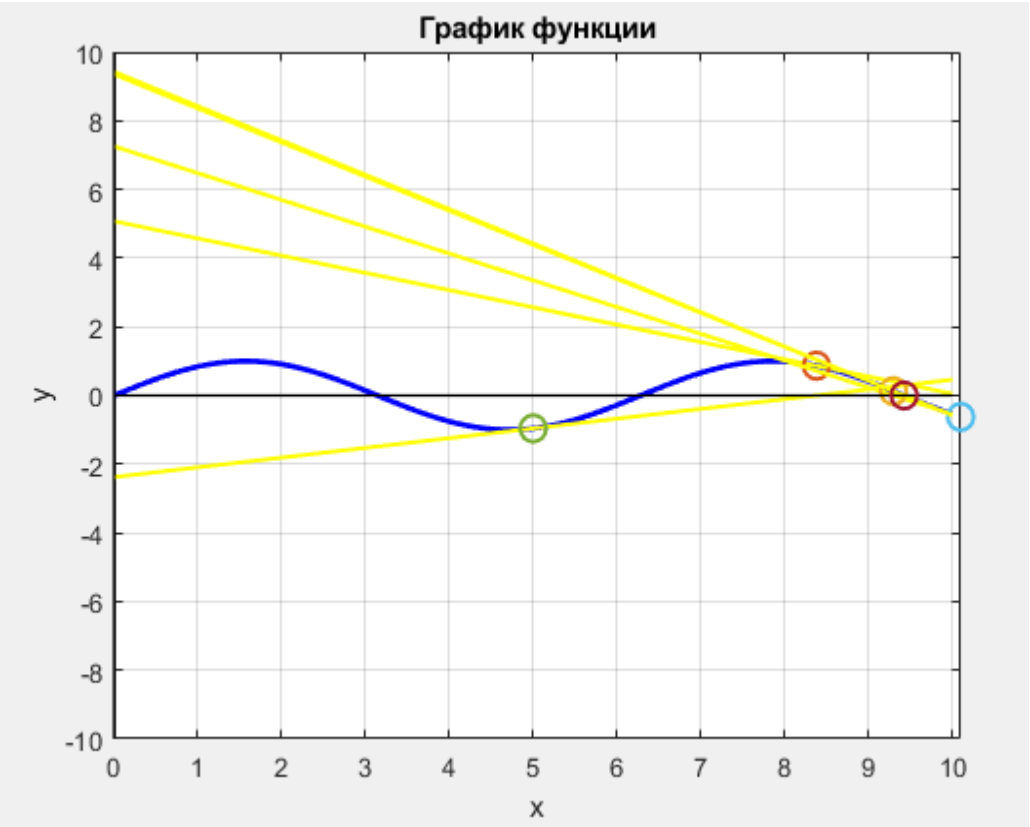


Рисунок 5 – График функции для вводимых данных

3. Интерполирование функции по формулам Ньютона.

3.1 Математическая постановка задачи интерполяции функции методом Ньютона

Пусть некоторая функция y = f(x) задана таблицей, т.е. для (n + 1)-го значения аргумента x известны (n + 1) значения функции y = f(x) (таблица 2).

x	x_0	x_1	…	x_n
f(x)	$f(x_0)$	$f(x_1)$	…	$f(x_n)$

Таблица 2 – Аргументы и их значения

Задача интерполяции заключается в замене функции y = f(x), аналитическое выражение которой неизвестно, другой функцией F(x), аналитическое выражение которой известно. Это дает возможность вычислять значения функции f(x) в точках, отличных от значений аргументов, фиксированных в таблице. При этом необходимо найти такое аналитическое выражение заданной функции f(x), т. е. функцию F(x), значения которой будут совпадать с табличными значениями функции y = f(x) для заданных значений аргументов x0, x1, …, xn:

$$F(x_0) = f(x_0) = y_0,$$
$$F(x_1) = f(x_1) = y_1,$$
$$\dots\dots\dots\dots\dots\dots \qquad (3.1)$$
$$F(x_n) = f(x_n) = y_n.$$

Функция F(x) называется интерполирующей функцией, точки x0, x1, …, xn – узлами интерполяции (узловыми точками), а задача нахождения функции F(x) – задачей интерполяции. Величина h = xk - xk-1 называется шагом интерполяции. Шаг интерполяции может быть постоянным или переменным.[2]

В некоторых случаях бывает необходимо строить функцию F(x) для функции y = f(x), аналитическое выражение которой известно, но является слишком сложным и неудобным для дальнейшего использования.[1]

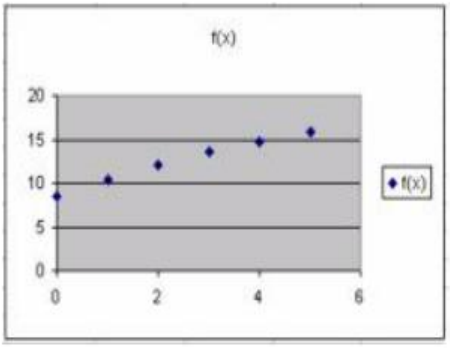
Геометрически задача интерполирования означает построение кривой, проходящей через точки плоскости с координатами (x_i, y_i) (рис.8). Чаще всего в качестве интерполирующей функции F(x) используются многочлены

$$P_n(x) = a_0x^n + a_1x^{n-1} + \ldots + a_{n-1}x + a_n \qquad (3.2)$$

В этом случае задача интерполяции состоит в подборе многочлена n- й степени $P_n(x)$, значения которого в узлах интерполяции совпадают со значениями интерполируемой функции y = f(x):

$$\begin{aligned} y_0 &= f(x_0) = P_n(x_0), \\ y_1 &= f(x_1) = P_n(x_1), \\ &\ldots\ldots\ldots\ldots\ldots\ldots \\ y_n &= f(x_n) = P_n(x_n) \end{aligned} \qquad (3.3)$$

и сводится к вычислению коэффициентов a_0, a_1, …, a_n интерполяционного многочлена. Доказано, что существует единственный интерполяционный многочлен n-ой степени, удовлетворяющий условиям (3.3).



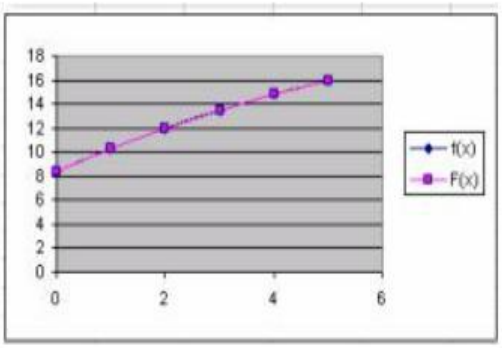


Рисунок 6 – Пример построения

3.2 Описание метода численной реализации задания

Интерполяция функции по формулам Ньютона - это метод численной аппроксимации функции, который позволяет приблизительно вычислить значение функции в точке, зная ее значения в некоторых других точках.

Формулы Ньютона основаны на интерполяционном многочлене Ньютона, который представляет собой многочлен, проходящий через заданные точки. Интерполяционный многочлен Ньютона может быть выражен в виде суммы членов, где каждый последующий член зависит от предыдущего.

Пусть дана функция f(x), заданная значениями в точках x_0, x_1, ..., x_n. Интерполяционный многочлен Ньютона для этой функции может быть записан в следующем виде:

$$P_n(x) = f(x_0) + f(x_0,x_1) + f(x_0,x_1,x_2)(x-x_1) + ... + f(x_0,x_1,\ldots x_n),...,x_n(x-x_1)...(x-x_{n-1}), \quad (3.4)$$

где $f(x_0)$, $f(x_0,x_1)$, $f(x_0,x_1,x_2)$, ..., $f(x_0,x_1,\ldots x_n)$ - разделенные разности, которые могут быть вычислены рекурсивно.

Для нахождения значения функции в точке x, необходимо вычислить интерполяционный многочлен $P_n(x)$. Значения разделенных разностей могут быть вычислены заранее, и затем используются для вычисления многочлена.

Одним из преимуществ формул Ньютона является то, что они могут быть использованы для интерполяции функций любой степени, и что они не требуют равномерного распределения точек. Однако, использование высоких степеней многочлена может привести к неустойчивости при вычислении, поэтому обычно рекомендуется использовать многочлены не более высокой степени 3-4.[2]

3.3 Описание ручного счета тестового примера предложенным методом

Пусть функция задана следующей таблицей (таблица 3):

x	y
1.215	0.106044
1.220	0.113276
1.225	0.119671
1.230	0.125324
1.235	0.130328
1.240	0.134776
1.245	0.138759
1.250	0.142367
1.255	0.145688
1.260	0.148809

Таблица 3 – Начальные данные

Определить значение функции y(x) при следующих значениях аргумента:

1) $x_1 = 1.2273$; 3) $x_2 = 1.253$;

2) $x_3 = 1.210$; 4) $x_4 = 1.2638$;

Составим таблицу конечных разностей (Таблица 4)

При составлении таблицы разностей ограничиваемся разностями третьего порядка, так как они практически постоянны.

Вычислим значения функции при x = 1,2273 и x=1,210 воспользуемся формулой Ньютона для интерполирования вперед:

$$y(x) \approx y_0 + t\Delta y_0 + \frac{t(t-1)}{2!}\Delta^2 y_0 + \frac{t(t-1)(t-2)}{3!}\Delta^3 y_0 + \ldots.. \quad (3.5)$$

где $t = \frac{(x-x_0)}{h}$

x_i	y_i	Δy_i	$\Delta^2 y_i$	$\Delta^3 y_i$
1.215	0.106044	0.007232	-0.000837	0.000095
1.220	0.113276	0.006395	-0.000742	0.000093
1.225	0.119671	0.005653	-0.000649	0.000093
1.230	0.125324	0.005004	-0.000556	0.000091
1.235	0.130328	0.004448	-0.000465	0.000090
1.240	0.134776	0.0003983	-0.000375	0.000088
1.245	0.138759	0.003608	-0.000287	0.000087
1.250	0.142367	0.003321	-0.000200	-
1.255	0.145688	0.003121	-	-
1.260	0.148809	-	-	-

Таблица 4 – Значения

1)Если x=1.2273, то примем x_0=1.225, тогда

$t = \frac{1.2273-1.225}{0.005} = 0.46$,

$y(1.2273) \approx 0.119671 + 0.46 * 0.005653 + \frac{0.46(-0.54)}{2}(-0.000649) + \frac{0.46(-0.54)(-1.54)}{6} 0.000093=$

$= 0.119671 + 0.0026004 + 0.0000806 + 0.0000059 = 0.1223579 \approx 0.122358$.

2)Если x=1.210, то примем x_0=1.215; тогда

$t = \frac{1.210-1.215}{0.005} = -1$

$y(1.210) \approx 0.106044 + (-1) * 0.007232 + \frac{(-1)(-2)}{2}(-0.000837) + \frac{(-1)(-2)(-3)}{6} 0.000095 =$
$= 0.097880.$

Для вычисления значений функции при x=1.253 и x=1.2638 воспользуемся формулой Ньютона для интерполяции назад:

$$y(x) \approx y_n + t\Delta y_{n-1} + \frac{t(t+1)}{2!}\Delta^2 y_{n-2} + \frac{t(t+1)(t+2)}{3!}\Delta^3 y_{n-3} + \ldots, \qquad (3.6)$$

где $t = \frac{(x-x_n)}{h}$

3)Если x = 1.253, то примем x_n = .1255; тогда

$t = \frac{1.252-1.255}{0.005} = -0.4$

$y(1.253) \approx 0.145688 + (-0.4)*0.0013284 + \frac{(-0.4)0.6}{2}(-0.000287) + \frac{(-0.4)0.6*1.6}{6} 0.000088 =$
$= 0.145688 – 0.0013284 + 0.0000344 – 0.0000056 = 0.1443884 \approx 0.144388.$

4)Если x = 1.2638, то примем x_n = 1.260, тогда

$t = \frac{1.2638-0.260}{0.005} = 0.76,$

$y(1.2638) \approx 0.1448809 + 0.76 * 0.003121 + \frac{0.76*1.76}{2}(-0.000200) + \frac{0.76*1.76*2.76}{6} 0.000087 =$
$= 0.148809 + 0.0023720 – 0.0001328 + 0.0000535 = 0.1511007 \approx 0.151101.$

3.4 Разработка алгоритма решения задачи

1. Задать значения аргументов и соответствующих им функций.
2. Разделить заданный интервал на равные промежутки.
3. Найти разделенные разности для всех значений функций на каждом промежутке.
4. Вычислить коэффициенты интерполяционного многочлена Ньютона, используя разделенные разности.
5. Составить интерполяционный многочлен Ньютона, используя найденные коэффициенты.
6. Проверить точность интерполяции, сравнивая значения функции, полученные с помощью интерполяционного многочлена Ньютона, с заданными значениями.
7. Применять интерполяционный многочлен Ньютона для вычисления значений функции в любых заданных точках на заданном интервале.[2]

3.5 Блок-схема

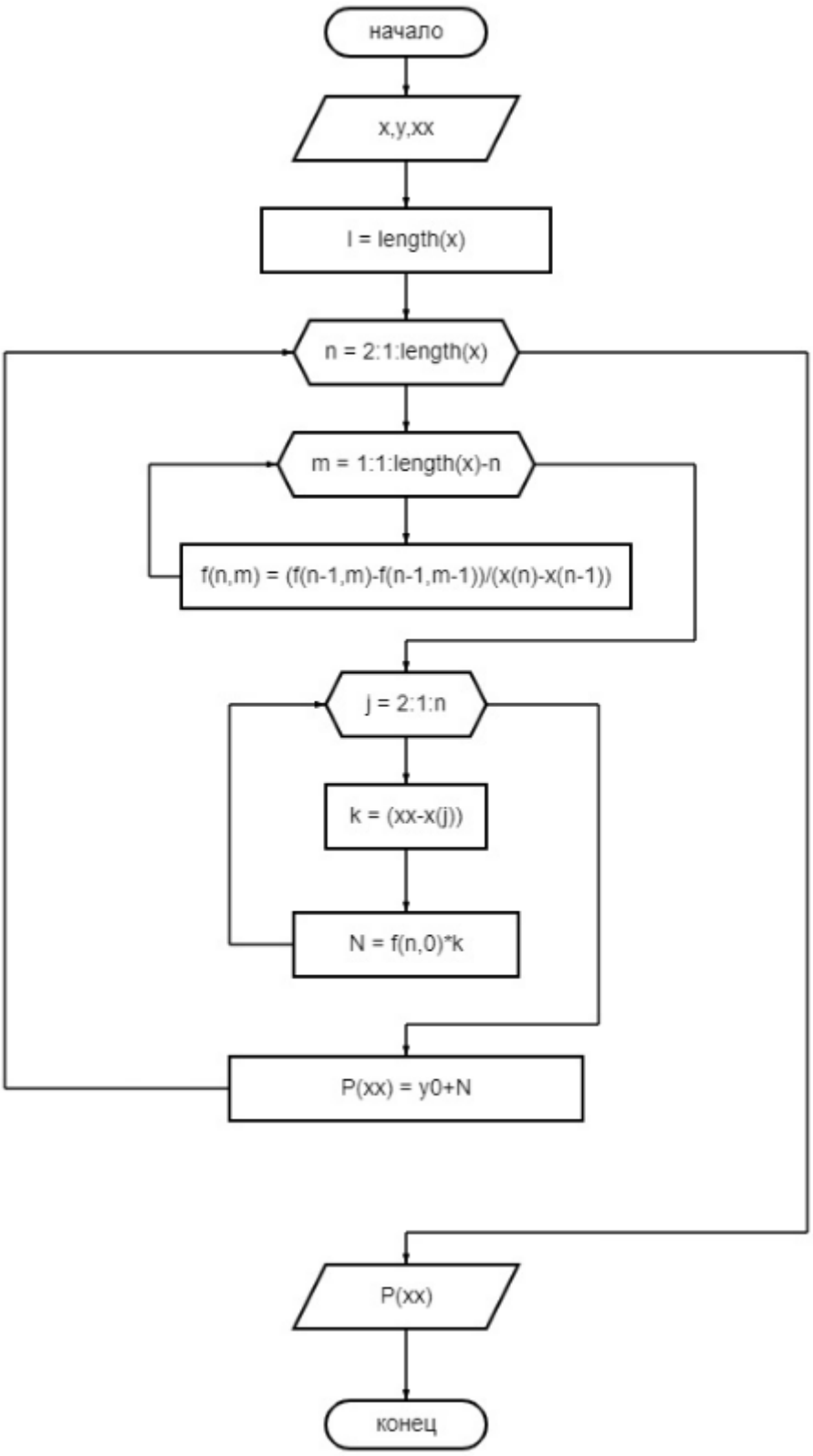


Рисунок 7 – Блок-схема

3.6 Результат работы программы

```
Command Window
Таблица значений x и y:
  x          y
1.000000   3.000000
2.000000   0.000000
3.000000   2.000000
4.000000   1.000000
5.000000   5.000000

Значения полинома в точках:
  xx          yy          yy(Matlab)
1.230000   0.870425   0.870425
6.000000   38.000000   38.000000

fx >>
```

Рисунок 8 – Вводимые данные и вывод

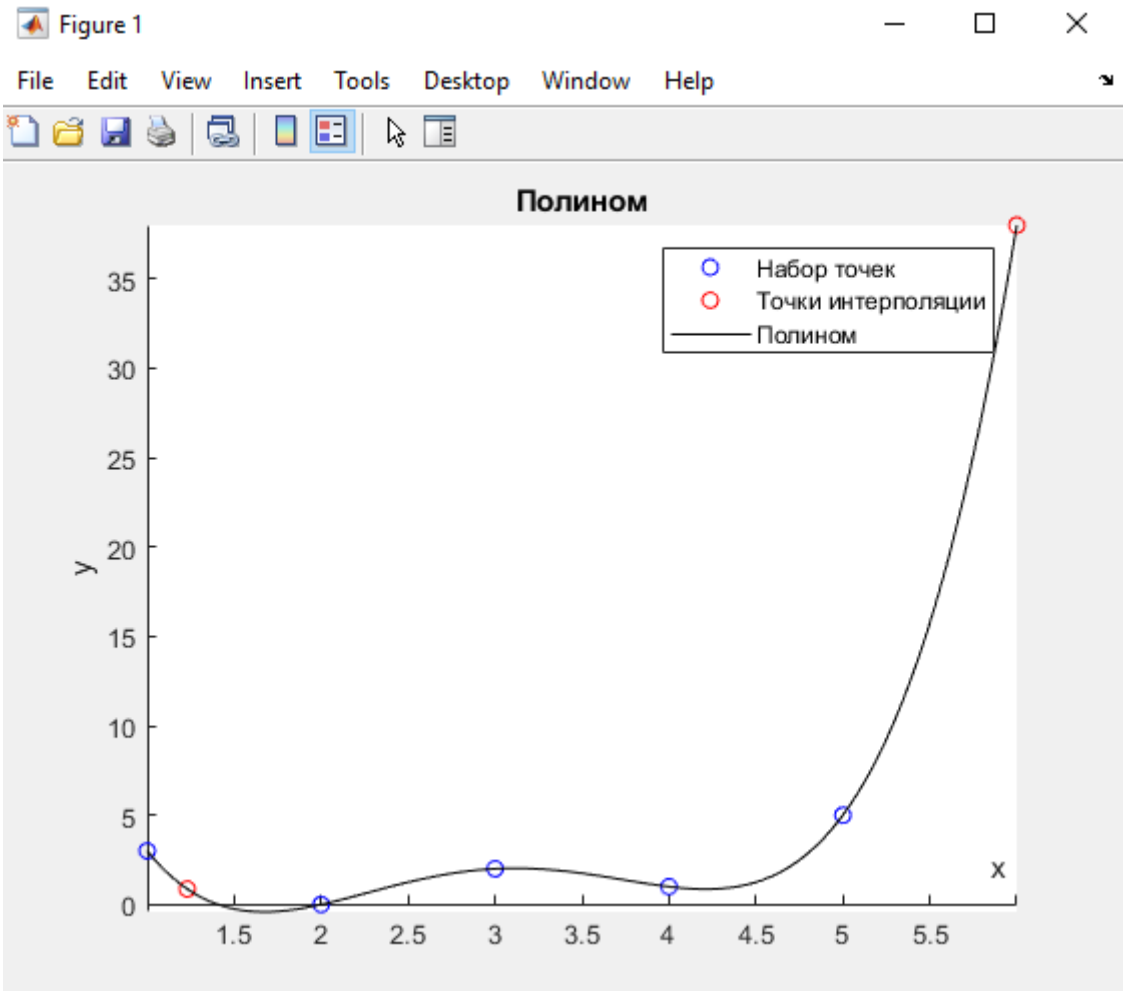


Рисунок 9 – График вводимых данных

4. Численное интегрирование методом Симпсона

4.1 Математическая постановка интегрирование методом Симпсона

Метод Симпсона является численным методом интегрирования функций и позволяет приближенно вычислить определенный интеграл от функции на заданном интервале.

Математическая постановка метода Симпсона выглядит следующим образом:

Пусть дана интегрируемая на отрезке [a, b] функция f(x). Разобьем отрезок [a, b] на n частей, где n является четным числом. Тогда шаг интегрирования h равен:

$$h = \frac{(b-a)}{2*n} \quad (4.1)$$

Точки разбиения отрезка [a, b] обозначим через $x_0, x_1, ..., x_n$, где $x_0 = a$, $x_n = b$, $x_i = a + i*h$. Тогда значение интеграла от f(x) на отрезке [a, b] можно приближенно вычислить по формуле:

$$\int_a^b f(x)dx \approx \frac{h}{3} * (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + ... + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)) \quad (4.2)$$

Здесь происходит интерполяция подынтегральной функции между каждой тройкой точек (x_{i-1}, x_i, x_{i+1}) на отрезках параболами, соединяющими эти точки. Таким образом, формула метода Симпсона основана на аппроксимации параболами подынтегральной функции и точнее приближает интеграл, чем трапеционный метод.[3]

4.2 Описание метода численной реализации задания

Чтобы реализовать метод Симпсона численно, нужно выполнить следующие шаги:

1. Разбить интервал интегрирования [a, b] на четное число отрезков (обычно n = 2, 4, 6, 8 и т.д.). Размер шага интегрирования определяется формулой (4.1):

2. Вычислить значения функции f(x) в точках $x_i = a + i*h$, где i = 0, 1, 2, ..., n.

3. Вычислить значения интеграла методом Симпсона, используя формулу (4.2):

4. Полученный результат будет приближенным. Чтобы увеличить точность, можно повторить шаги 1-3 с более мелким шагом интегрирования, и затем объединить результаты, например, используя метод Ричардсона.[4]

4.3 Описание ручного счета тестового примера предложенным методом

Вычислить приближенно определенный интеграл по формуле Симпсона с точностью до 0,001. Разбиение начать с двух отрезков 2n = 2

I = $\int_{1,2}^{2} \sqrt{1 + 2x^2 - x^3} dx$

Решение:

Тип задания – необходимо вычислить определенный интеграл с определенной точностью. Существует формула, которая сразу позволит определить нужное количество отрезков чтобы гарантированно достичь требуемой точности. Придётся находить четвертую производную и решать экстремальную задачу.[3]

Начинаем решать. Если у нас два отрезка разбиения 2n=2, то узлов будет на один больше: x_0,x_1,x_2.

Формула Симпсона принимает весьма компактный вид:

$$\int_a^b f(x)dx \approx \frac{h}{3}(f(x_0) + f(x_2) + 4f(x_1)) \quad (4.3)$$

Вычислим шаг разбиения: h = $\frac{(b-a)}{2n} = \frac{(2-1,2)}{2} = \frac{0,8}{2} = 0,4$

Заполним расчетную таблицу:

i	0	1	2
x_i	1,2	1,6	2
$f(x_i)$	1,466970	1,422674	1

Таблица 5 - Расчетная таблица для метода Симпсона по двум отрезкам разбиения

Как заполняется таблица:

В верхнюю строку записываем «счётчик» индексов.

Во второй строке сначала пишем нижний предел a = x_0 = 1,2 интегрирования, а затем последовательно приплюсовываем шаг h = 0,4.

В третью строку заносим значения подынтегральной функции. Например, если x_1 = 1,6, то $f(x_1) = \sqrt{1 + 2 * (1,6)^2 - (1,6)^3} \approx 1,422670$. Сколько оставлять знаков после запятой? Действительно, в условии опять об этом ничего не сказано. Принцип таков, смотрим на требуемую точность: 0,001. И прибавляем дополнительно 2-3 разряда. То есть, округлять нужно до 5-6 знаков после запятой.[5]

В результате:

$\int_{1,2}^{2}\sqrt{1+2x^2-x^3}dx \approx \frac{0,4}{3}(1,466970+1+4*1,422674) \approx 1,087689$

Первичный результат получен. Теперь удваиваем количество отрезков до четырёх: 2n=4. Формула Симпсона для данного разбиения принимает следующий вид:

Формула Симпсона для четырёх отрезков разбиения 2n=4

$$\int_a^b f(x)dx \approx \frac{h}{3}(f(x_0)+f(x_4)+2f(x_2) + 4(f(x_1)+f(x_3))) \qquad (4.4)$$

Вычислим шаг разбиения:

h = $\frac{(b-a)}{2n} = \frac{(2-1,2)}{4} = \frac{0,8}{4} = 0,2$

Заполним расчетную таблицу:

i	0	1	2	3	4
x_i	1,2	1,4	1,6	1,8	2
$f(x_i)$	1,466970	1,475127	1,422674	1,283745	1

Таблица 6 - Расчетная таблица для метода Симпсона по четырём отрезкам разбиения

Таким образом:

$\int_{1,2}^{2}\sqrt{1+2x^2-x^3}dx \approx \frac{0,2}{3}(1,466970+1+2*1,422674 + 4*(1,475127 + + 1,283745)) \approx 1,089854$

Найдём абсолютное значение разности между приближениями:

$|I_4 - I_2| \approx |1,089854 - 1,087689| = 0,002165$

Правило Рунге для метода Симпсона очень полезное.

$\frac{1}{15}|I_4 - I_2| \approx \frac{1}{15} * 0,002165 = 0,000144 < 0,001$, точность здесь не страдает:

I = $\int_{1,2}^{2}\sqrt{1+2x^2-x^3}dx \approx I_4 \approx 1,089854 \approx 1,090$

Ответ: I = $\int_{1,2}^{2}\sqrt{1+2x^2-x^3}dx \approx 1,090$ с точностью до 0,001.

4.4 Разработка алгоритма решения задачи

Алгоритм метода Симпсона для нахождения интеграла функции f(x) на интервале [a, b]:

1. Разбиваем отрезок интегрирования [a, b] на четное число n интервалов равной длины $h = \frac{(b-a)}{2n}$

2. Вычисляем значения функции f(x) в узлах сетки $x_0 = a$, $x_1 = a+h$, $x_2 = a+2h$, ..., $x_n = b$.

3. Для каждой пары узлов x_i и x_i+1 вычисляем значение интеграла на отрезке $[x_i, x_{i+1}]$ по формуле Симпсона:

$$I_i = \frac{h}{3} * (f(x_i) + 4 * f(x_i + h) + f(x_i + 2h)) \quad (4.5)$$

4. Суммируем значения Ii для всех пар узлов:

$I = I_0 + I_1 + I_2 + ... + I_{n-1}$

5. Полученная сумма I является приближенным значением интеграла от функции f(x) на отрезке [a, b].

6. Если требуется увеличить точность вычислений, можно уменьшить шаг h, повторив алгоритм для каждой новой сетки.[6]

4.5 Блок-схема

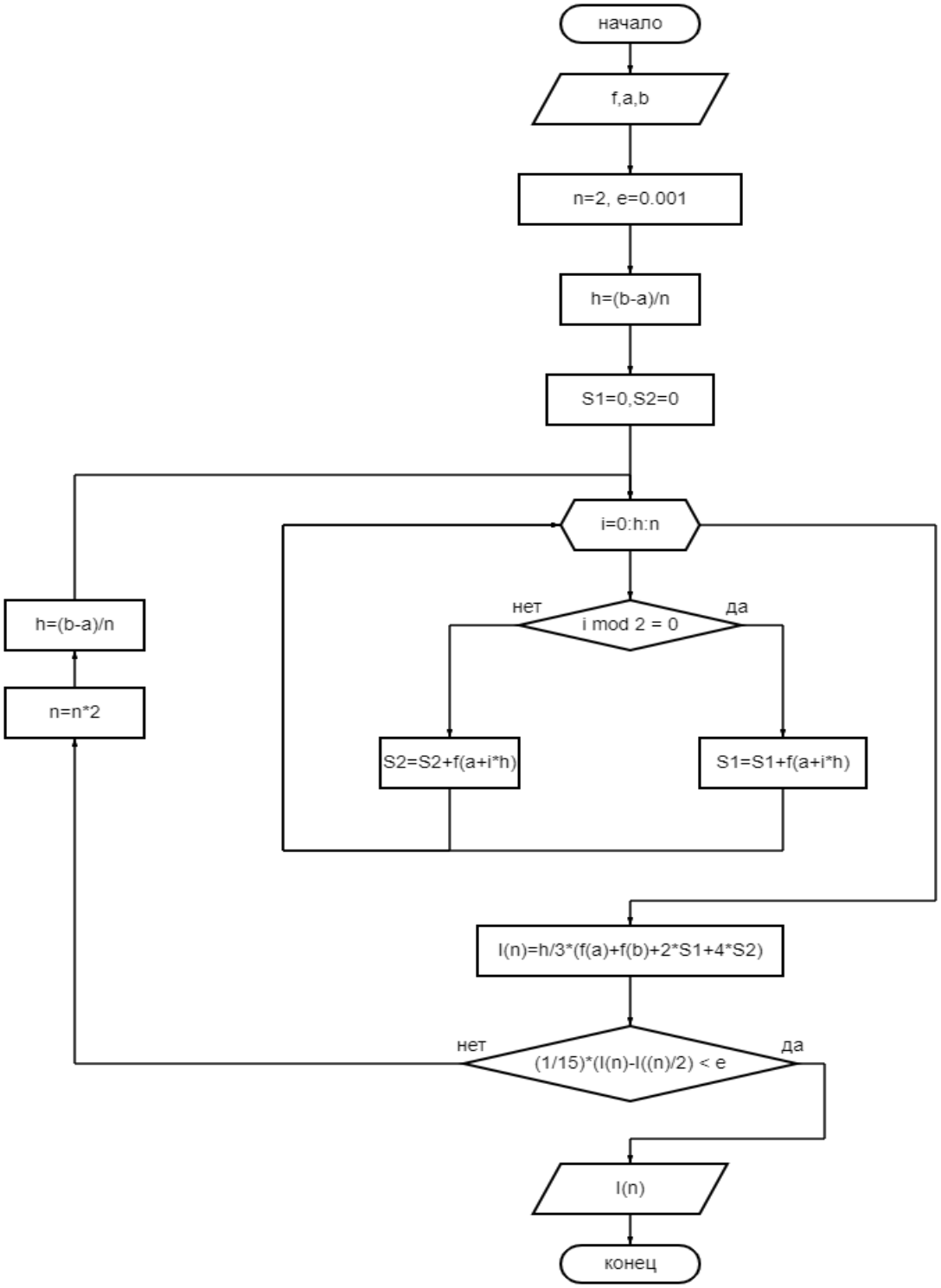


Рисунок 10 – Блок-схема

4.6 Результат работы программы

```
Command Window
Введите функцию: x
Введите нижнюю часть интеграла: 0
Введите верхнюю часть интеграла: 10
Введите точность: 0.001
Интеграл пользовательским методом = 50.000
Интеграл встроенной функцией integral = 50.000
Погрешность интеграла = -0.000
fx >>
```

Рисунок 11 – Входные данные и вывод

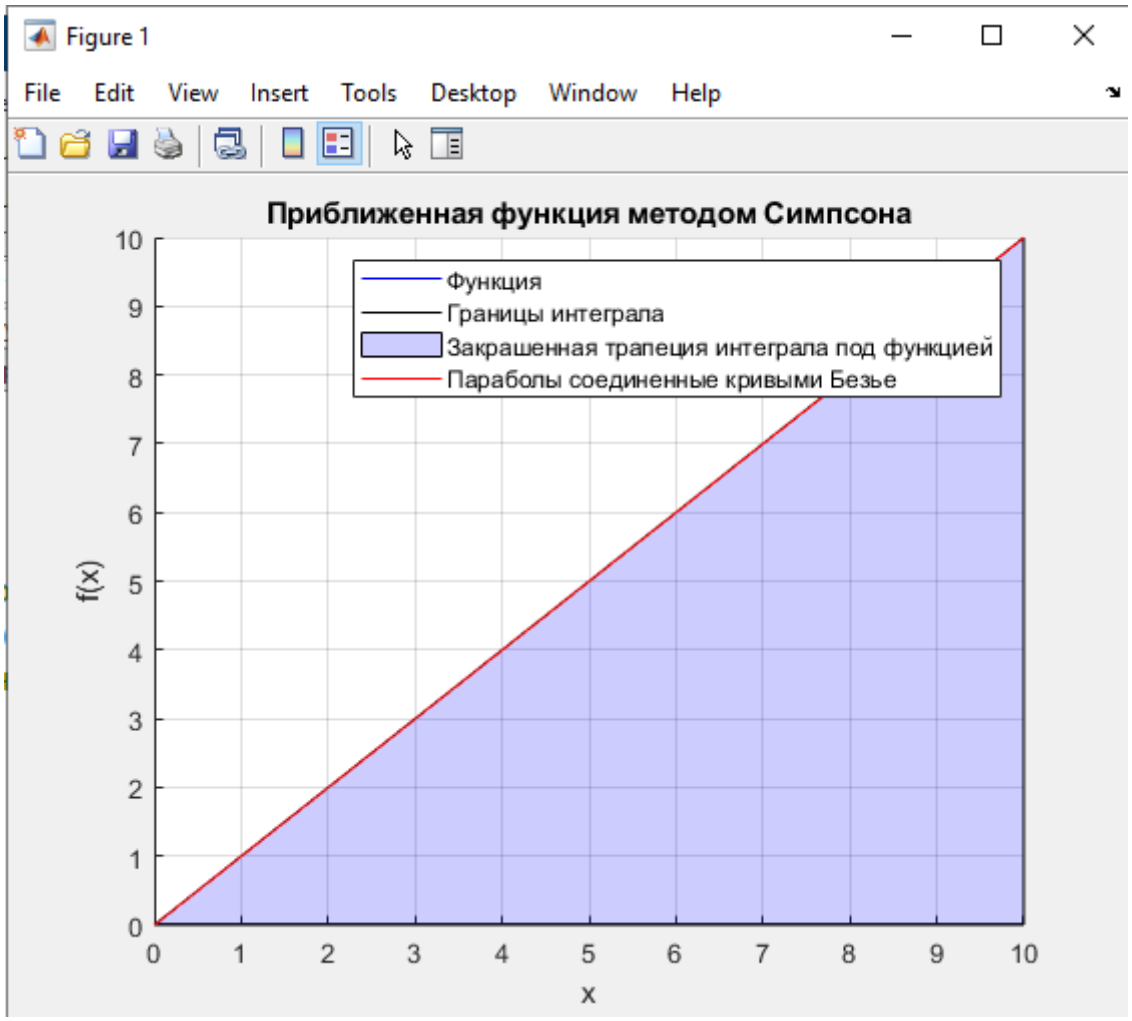


Рисунок 12 – График входных данных

5. Работа программы, виды неправильного и правильного ввода пользователем

При вводе пользователем в главном меню буквы или не существующего пункта будет выдаваться ошибка:

```
Command Window
Главное меню
Эта программа разработана для решения трёх численных методов
Выберите пункт меню
1. Решение нелинейного уравнения методом Ньютона(касательные)
2. Интерполирование функции по формулам Ньютона
3. Численное интегрирование методом Симпсона
4. Информация об авторе
5. Выход
Выберите номер пункта программы 6
Некоректно введен пункт.
Выберите номер пункта программы f
Некоректно введен пункт
fx Выберите номер пункта программы
```

Рисунок 13 – Некорректный ввод

Корректно будет ввести цифру из тех, которыми помечены пункты меню. На примере введем цифру “1”.

```
Command Window
Главное меню
Выберите пункт меню
1. Демонстрационный пример
2. Работа с данными пользователя
3. Информация об авторе
4. Выход
fx Выберите номер пункта программы
```

Рисунок 14 – Корректный ввод и вывод первого пункта

И так, при вводе значения “1”, мы получаем нашу под программу, в которой так же есть меню. При вводе неправильного значения в меню вы уже ознакомились выше. Поэтому введем значение “1”.

```
Command Window
Программа для решения уравнения a*ln(x) + b*x^2 + c = 0 методом Ньютона
Автор: Панчеко Андрей
Цель: Показать работу метода Ньютона
Метод сошелся за 4 итераций
Решено с точностью:
Корень уравнения: 0.497150
Эта программа была написана Панченко Андреем для демонстрации метода Ньютона
fx >>
```

Рисунок 15 – Ввод корректного значения и вывод работы программы

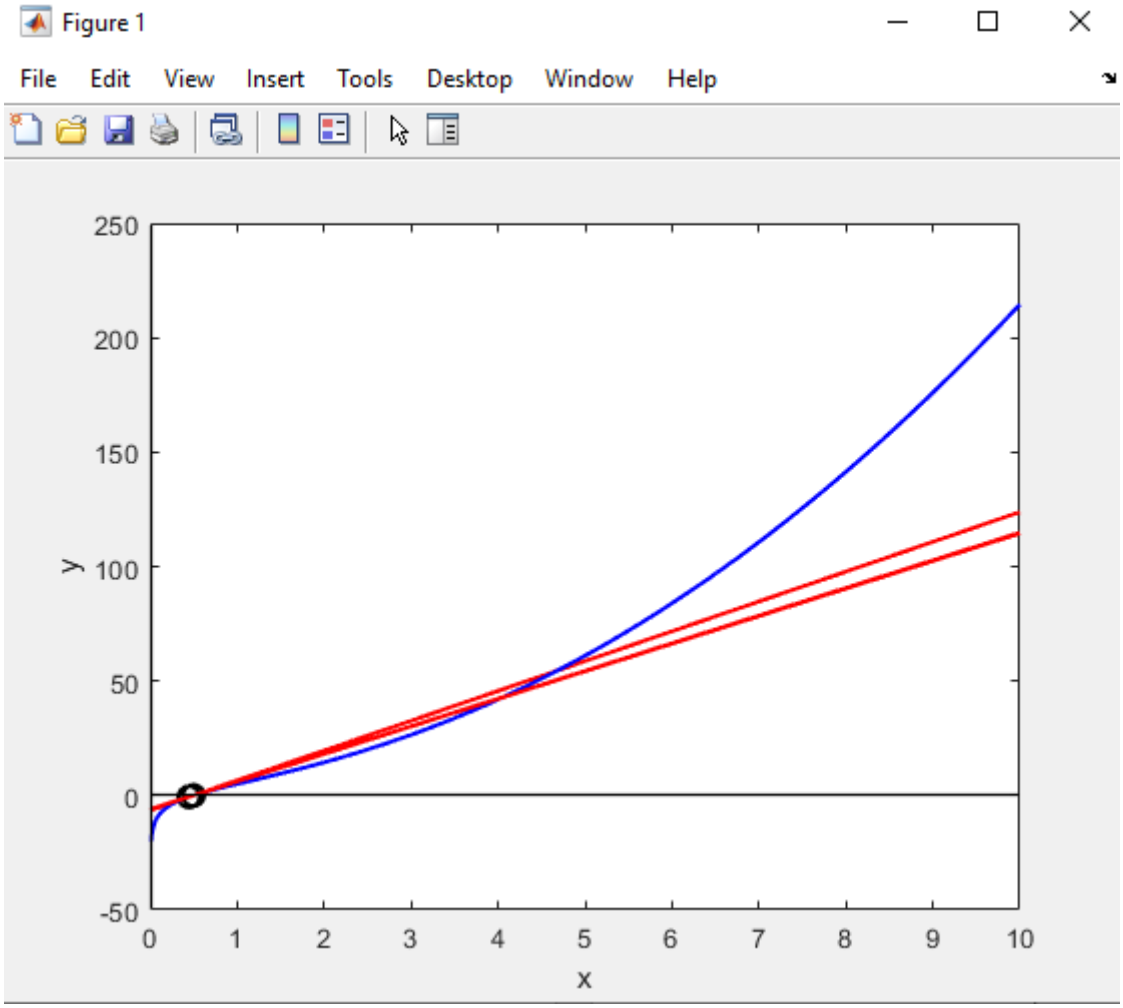


Рисунок 16 – График демонстрационного примера

Здесь нам показывается демонстрационный пример решения заданным методом. Вернемся в меню указанном выше на Рисунке 14 и выберем пункт “2”.

```
Command Window
Введите функцию: san(x)
Undefined function 'san' for input arguments of type 'double'.
Введите функцию: f
Unrecognized function or variable 'f'.
Введите функцию: 0,0,0,
Invalid expression. Check for missing multiplication operator, missing or unbalanced delimiters, or other syntax error. To construct matrices,
fx Введите функцию:
```

Рисунок 17 – Ввод функции

Как мы видим на рисунке выше, вводя некорректную функцию пользователь получает сообщение о некорректном вводе функции и просьбой ввести заново. Введем верную функцию.

```
Command Window
Введите функцию: sin(x)
Введите левое значение оси x: f
Левая граница должна быть числом, в числе должен быть один разделитель.
Введите левое значение оси x: 0,0,
Левая граница должна быть числом, в числе должен быть один разделитель.
fx Введите левое значение оси x:
```

Рисунок 18 – Некорректный ввод границ

На Рисунке 18 показан ввод правильной функции, но не правильный ввод границ функции и запросом ввести заново. Введем их корректно.

```
Введите функцию: sin(x)
Введите левое значение оси x: 0
Введите правое значение оси x: 7
fx Введите начальное приближенное значение x0, используя точку для разделения дробной части:
```

Рисунок 19 – Корректный ввод значений пользователем

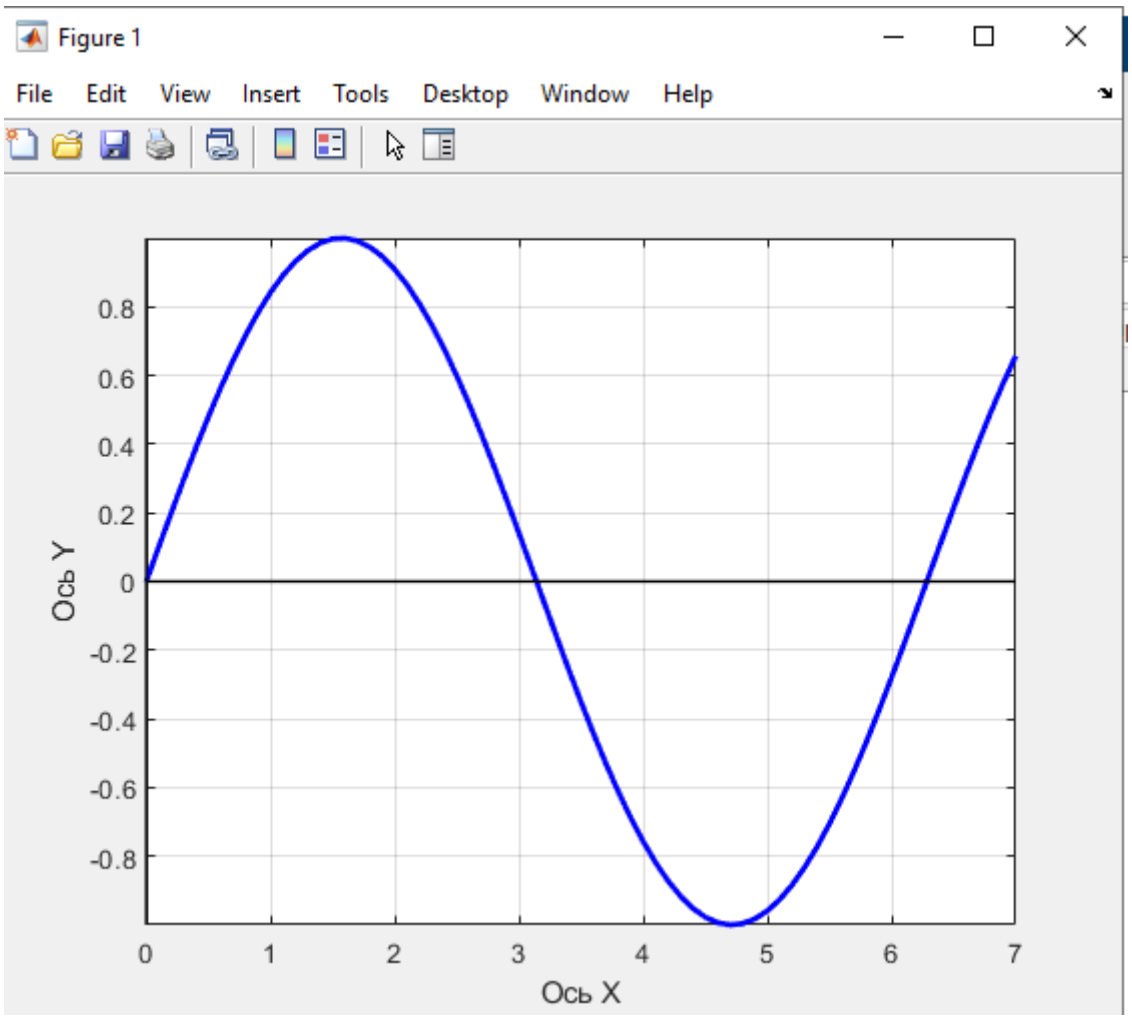


Рисунок 20 – График перед вводом начального значения и точности

Введя корректные значения, мы получаем график перед вводом начального приближенного значения.

```
Введите начальное приближенное значение x0, используя точку для разделения дробной части: f
x0 должно быть числом, в числе должен быть один разделитель.
Введите значение x0: 0,0,0,0,
x0 должно быть числом, в числе должен быть один разделитель.
Введите значение x0:
```

Рисунок 21 – Неверный ввод

На Рисунке 21 показан неверный ввод начального приближенного значения. Покажу верный ввод.

```
Введите значение x0: 1
Введите точность вычисления корня e, используя точку для разделения дробной части: f
e должно быть числом, в числе должен быть один разделитель.
Введите точность вычисления корня e, используя точку для разделения дробной части: 0,0,0,0
e должно быть числом, в числе должен быть один разделитель.
Введите точность вычисления корня e, используя точку для разделения дробной части:
```

Рисунок 22 – Корректный ввод x0

На Рисунке 22 показан верный ввод приближенного значения и запросом ввода точности. На примере показан некорректный ввод точности, покажу корректный.

```
Введите точность вычисления корня e, используя точку для разделения дробной части: 0.001
Уравнение решено за 4 итераций
Корень уравнения: -0.000096
Корень уравнения по встроенной функции fzero: 0.000000
fx >>
```

Рисунок 23 – Корректный ввод точности и результат работы программы

Так мы ввели верную точность и нам вышел результат работы программы с графиком ниже.

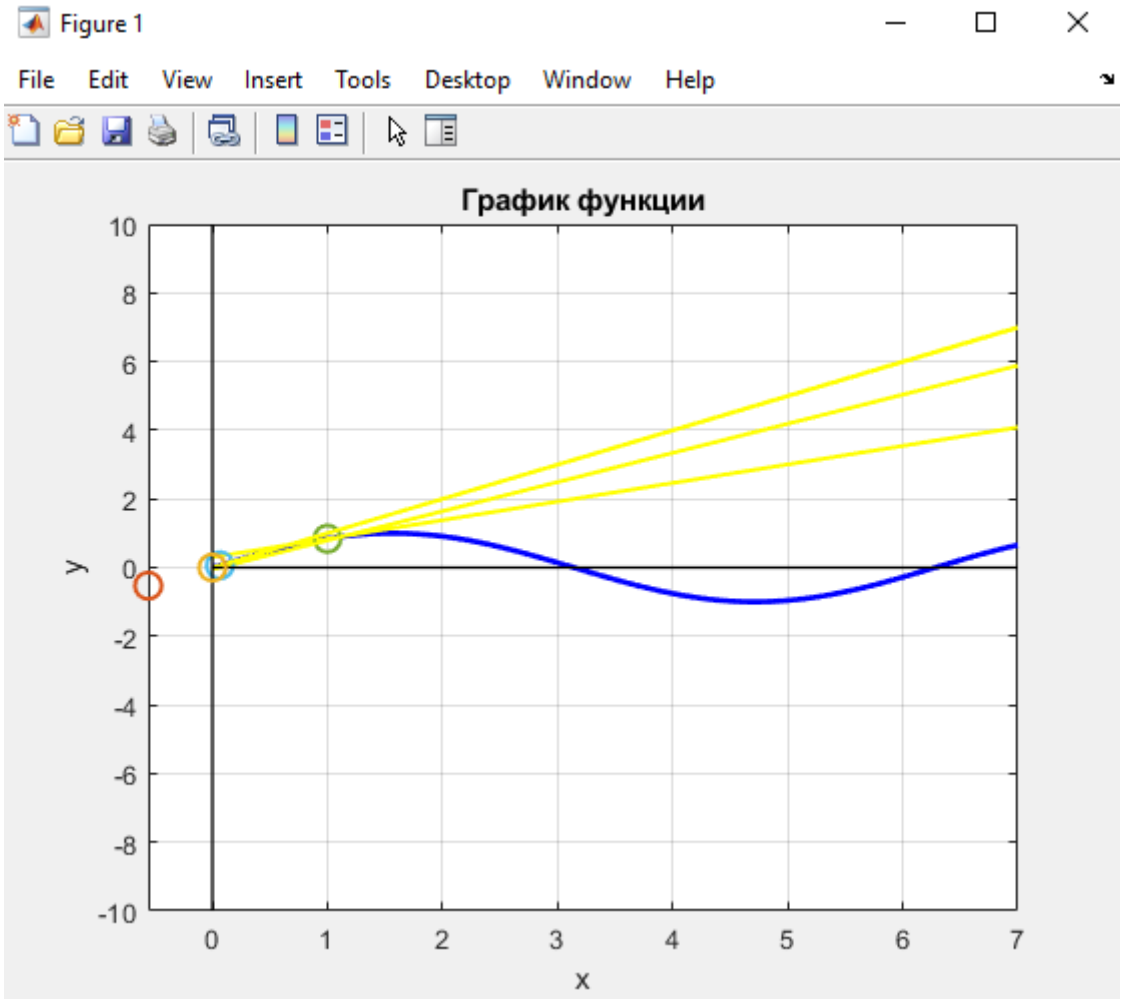


Рисунок 24 – График работы программы

Возвращаясь в меню программы выберем цифру “3”.

```
Command Window
Программа для решения уравнения методом Ньютона.
Автор: Панченко Андрей.
fx >>
```

Рисунок 25 – Информация об авторе

Нам выходит информация об авторе. Выбирая пункт меню “4” программа завершается.

```
Command Window
Работа программы завершена
fx >>
```

Рисунок 26 – Конец программы

Возвращаясь к Рисунку 13 введем значение “2”.

```
Command Window
Главное меню
Выберите пункт меню
1. Демонстрационный пример
2. Работа с данными пользователя
3. Информация об авторе и программе
4. Выход
fx Выберите номер пункта программы:
```

Рисунок 27 – Меню программы второго пункта

Нам выходит меню программы второго пункта и тут все так же, как и в прошлых примерах неправильного ввода значений, смотрите Рисунок 13. Введем значение “1”.

```
Command Window
Таблица значений x и y:
  x          y
1.200000   10.247000
1.500000   10.541000
1.700000   10.815000
2.000000   11.089000
2.400000   11.357000
2.600000   11.621000
3.100000   11.896000
3.300000   12.164000
3.500000   12.430000
3.900000   12.708000

Значения полинома в точках:
  xx          yy         yy(Matlab)
1.250000   10.466652   10.466652
3.250000   12.078408   12.078408
```

Рисунок 28 – Вывод демонстрационного примера

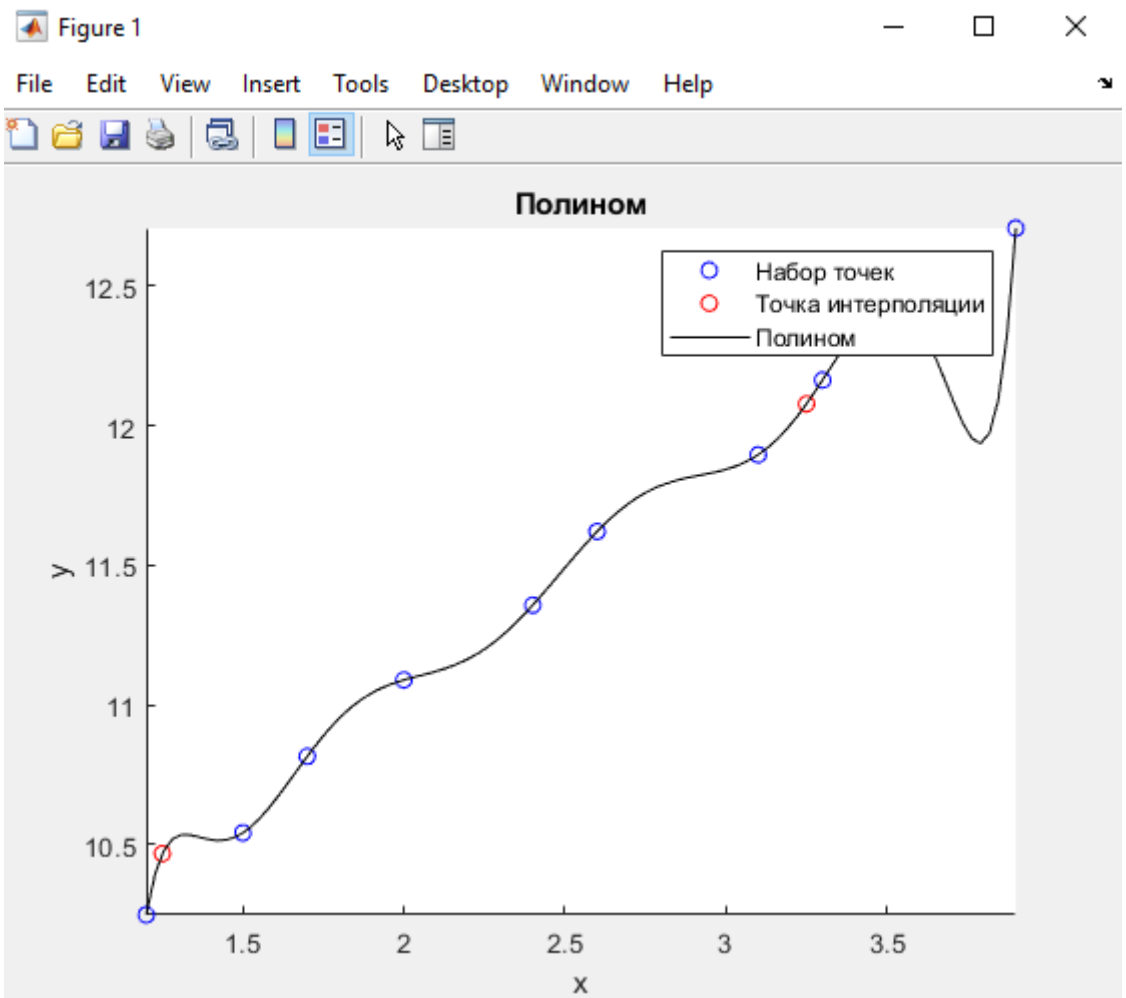


Рисунок 29 – График функции

Нам выходят начальные значения, вывод работы программы, и график. Вернемся к Рисунку 27 и введем “2”.

```
Command Window
Введите значения массива x, разделяя их пробелами: f
Массив x должен содержать не менее двух уникальных чисел, разделенных пробелами и не буквы.
Введите значения массива x: 0,0,,00,
Массив x должен содержать не менее двух уникальных чисел, разделенных пробелами и не буквы.
Введите значения массива x:
```

Рисунок 30 – Запрос ввода массива значений x

При вводе буквы или некорректных цифр запрашивает заново ввод значений. Покажем правильный пример.

```
Введите значения массива x: 1 5 -1 3
Введите значения массива y, разделяя их пробелами: f
Массив y должен содержать столько же чисел, сколько и массив x, разделенных пробелами и не буквы.
Введите значения массива y: 0 2
Массив y должен содержать столько же чисел, сколько и массив x, разделенных пробелами и не буквы.
Введите значения массива y:
```

Рисунок 31 – Правильный ввод значений массива x и неправильно массива y

Мы ввели правильные значения в массив x и y нас запросило ввести значения y. Мы введи некорректно, и оно запросило заново ввести значения. Введем правильно.

```
Введите значения массива y: 0 1 3 2
Введите значения xx, разделяя их пробелами или запятыми и используя точку для разделения дробной части: f
xx должно быть числом, в числе должен быть один разделитель.
Введите значения xx: 1
Введите значения xx, которые не совпадают с значениями в массиве x.
Введите значения xx:
```

Рисунок 32 – Корректный ввод массива y и некорректный ввод точки xx

Так мы ввели корректные значения массива y и y нас запросило ввести точку, которую пользователь хочет найти с помощью этих массивов. Мы ввели неверно. Покажем, как нужно вводить верно.

```
Command Window
Таблица значений x и y:
  x          y
-1.000000   3.000000
1.000000   0.000000
3.000000   2.000000
5.000000   1.000000

Значения полинома в точках:
  xx          yy          yy(Matlab)
1.500000   0.343750   0.343750

fx >>
```

Рисунок 33 – Ввод корректного значения xx и результат программы

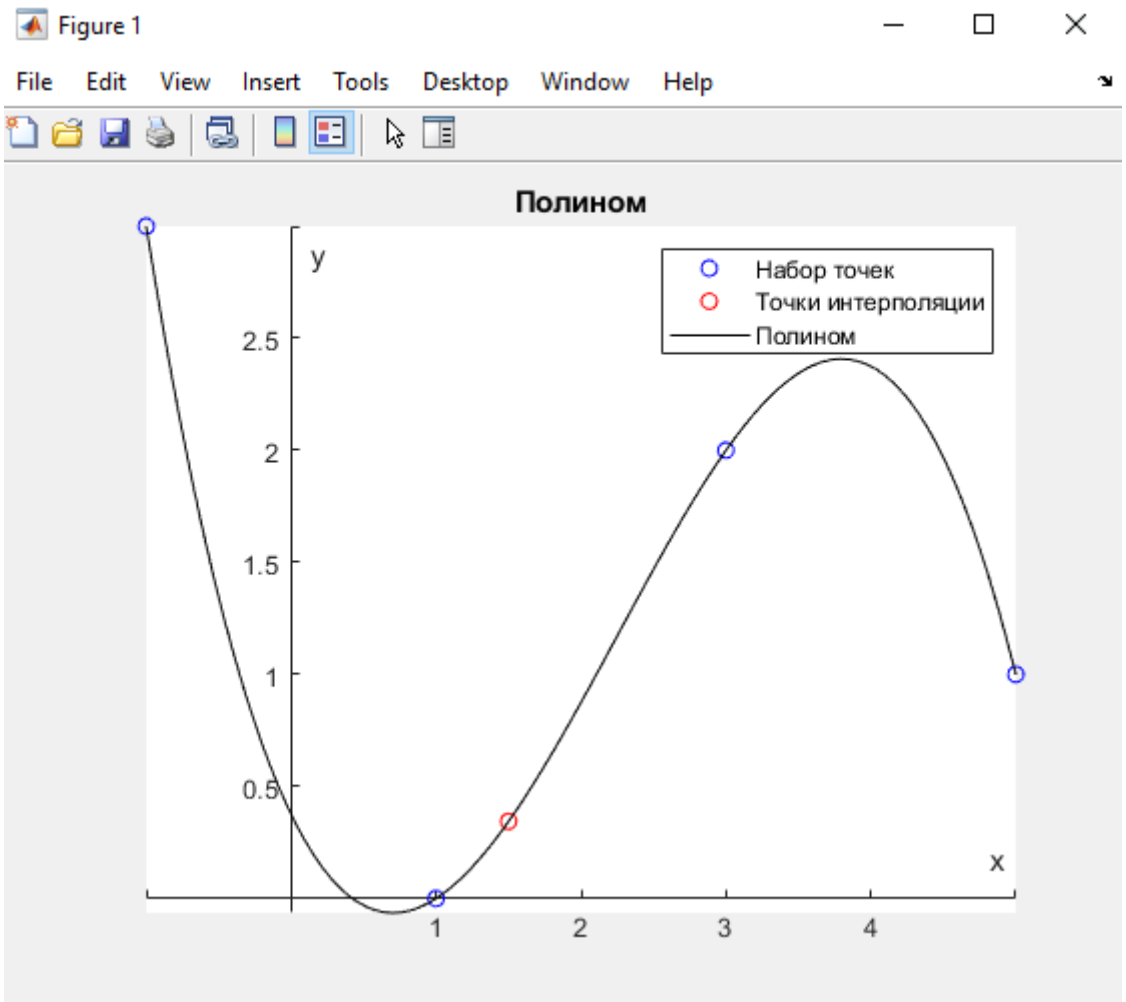


Рисунок 34 – График функции

Мы ввели корректное значение xx и нам вышел результат программы и график функции. Возвращаемся к Рисунку 27 и выбираем пункт “3”.

```
Command Window
Интерполирование функции по формулам Ньютона.
Автор: Панченко Андрей.
fx >>
```

Рисунок 35 – Информация об авторе

Так нам выводится информация о программе и об авторе. Возвращаясь к Рисунку 27 выберем пункт меню “4”.

```
Command Window
Работа программы завершена
fx >>
```

Рисунок 36 – Завершение программы

Таким образом, программа завершается. Возвращаемся к Рисунку 13 и выбираем пункт “3”.

```
Command Window
Главное меню
Выберите пункт меню
1. Демонстрационный пример
2. Работа с данными пользователя
3. Информация об авторе и программе
4. Выход
fx Выберите номер пункта программы:
```

Рисунок 37 – Меню пункта 3

Нам выходит меню пункта “3” где точно так же можно ввести некорректные значения как мы рассматривали до этого. Выберем пункт “1”.

```
Command Window
Функция: exp.^(-(x.^2./2))
Нижнее значение интеграла = 0
Верхнее значение интеграла = 0.610
Точность = 0.001
Интеграл пользовательским методом = 0.574
Интеграл встроенной функцией integral = 0.574
Погрешность интеграла = -0.000
fx >>
```

Рисунок 38 – Демонстрационный пример

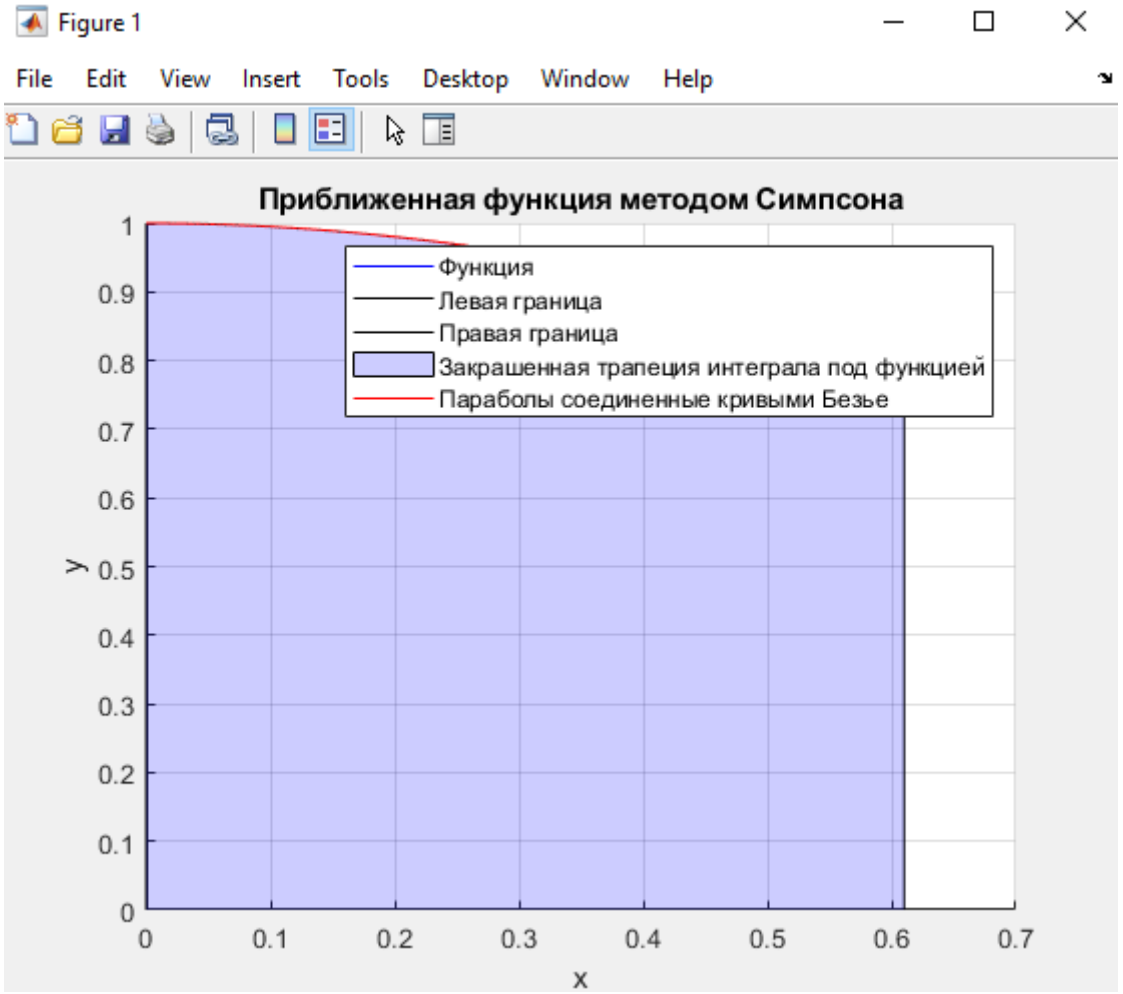


Рисунок 39 – График функции

Нам выходит демонстрационный пример с графиком функции. Возвращаясь к Рисунку 37 выберем пункт “2”.

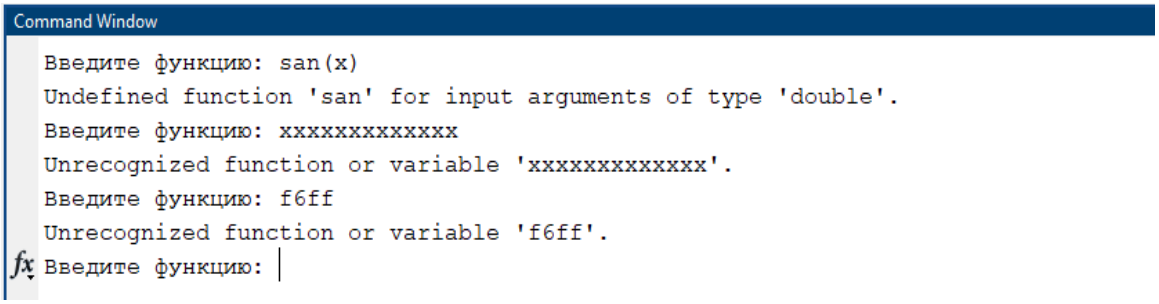


Рисунок 40 – Ввод функции

Нас запрашивает ввести функцию. В этом примере мы ввели неверную функцию. Покажем, как надо.

```
Введите функцию: sin(x)
Введите нижнюю часть интеграла: f
Должно быть числом, в числе должен быть один разделитель.
Введите нижнюю часть интеграла: 0,0,
Должно быть числом, в числе должен быть один разделитель.
fx Введите нижнюю часть интеграла:
```

Рисунок 41 – Верный ввод функции и неверный ввод нижней части интеграла

Таким образом, мы ввели верную функцию и у нас запросило ввести нижнюю часть интеграла. В нашем случае мы ввели неверное значение, покажем, как нужно правильно.

```
Введите нижнюю часть интеграла: 0
Введите верхнюю часть интеграла: f
Должно быть числом, в числе должен быть один разделитель.
Введите верхнюю часть интеграла: 0,0,
Должно быть числом, в числе должен быть один разделитель.
fx Введите верхнюю часть интеграла:
```

Рисунок 42 – Корректный ввод нижней части интеграла, но не верно верхний

И так, мы ввели верное значение нижней части интеграла и нас запросило ввести верхнюю часть. В нашем случае мы ввели неверно, покажем, как верно.

```
Введите верхнюю часть интеграла: 5
Введите точность: f
Должно быть числом, в числе должен быть один разделитель.
Введите точность: 0,0,0
Должно быть числом, в числе должен быть один разделитель.
fx Введите точность:
```

Рисунок 43 – Верный ввод верхнего значения и неверный ввод точности

Мы ввели верное верхнее значение и нас запросило ввести точность. В нашем случае точность введена некорректно, поэтому мы покажем, как верно.

```
Введите точность: 0.001
Интеграл пользовательским методом = 0.716
Интеграл встроенной функцией integral = 0.716
Погрешность интеграла = -0.000
fx >>
```

Рисунок 44 – Корректный ввод точности и результат работы программы

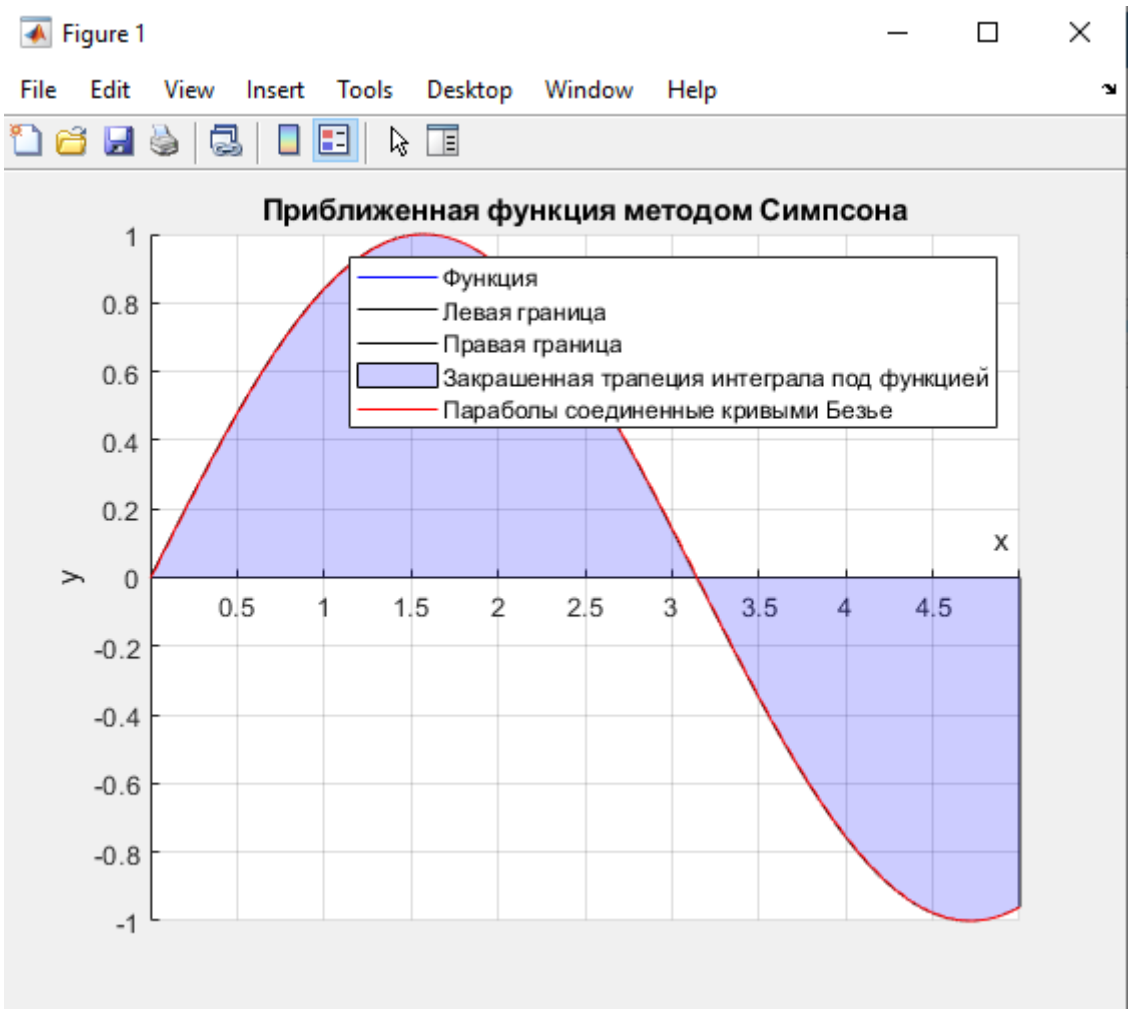


Рисунок 45 – График функции

Мы ввели корректную точность и получили результат работы программы и график функции. Возвращаясь к Рисунку 37 выбираем пункт “3”.

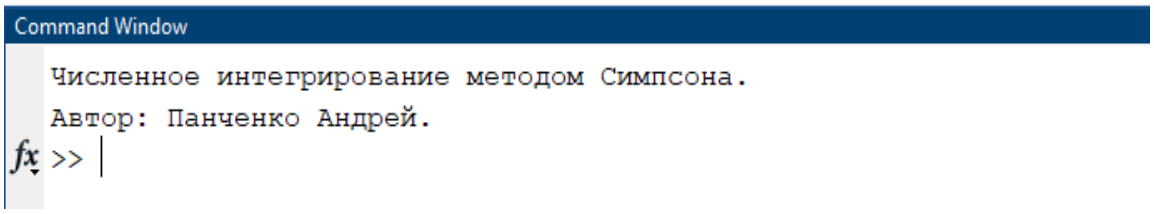


Рисунок 46 – Информация о программе и авторе

Нам выходит информация об авторе. Возвращаясь к Рисунку 37 выбираем пункт “4”.

Рисунок 47 – Завершение работы программы

У нас завершилась работа программы пункта “3”. Возвращаясь к Рисунку 13 выберем пункт меню “4”.

```
Command Window
Автор: Панченко Андрей.
Делал долго, но получилось.
Номер телефона и почту не дам ^_^.
fx >>
```

Рисунок 48 – Информация об авторе

Нам выходит информация об авторе. И возвращаясь к Рисунку 13 выберем пункт “5”.

```
Command Window
Работа программы завершена
fx >>
```

Рисунок 49 – Выход из программы

Мы завершаем работу программы. Таким образом, мы рассмотрели все варианты правильного и неправильного ввода данных пользователем и их решение программой.

Заключение

В ходе изучения решения нелинейных уравнений методом Ньютона (касательных), интерполирования функций по формулам Ньютона и численного интегрирования методом Симпсона, мы узнали о различных алгоритмах численного анализа, которые могут применяться для приближенного решения математических задач.

Метод Ньютона для решения нелинейных уравнений позволяет найти корни уравнения с высокой точностью, используя последовательность приближений, получаемых через итерационный процесс. Этот метод является очень эффективным, но требует некоторых условий для сходимости.

Интерполяция функций по формулам Ньютона позволяет аппроксимировать функцию на отрезке [a,b] с помощью многочлена n-й степени. Данный метод находит применение в различных областях, таких как приближенное вычисление значений функций и построение графиков.

Метод Симпсона является одним из наиболее точных методов численного интегрирования, позволяющим вычислять определенный интеграл на отрезке [a,b] с высокой точностью. Он основан на аппроксимации интеграла параболами, проходящими через три точки на отрезке [a,b].

Таким образом, изучение этих методов позволяет решать широкий круг задач, связанных с приближенным вычислением функций и интегралов, а также решением нелинейных уравнений. Они находят применение в различных областях, таких как физика, математика, экономика, и т.д.

Список использованных источников

1. Кудрявцев Л. Д. Курс математического анализа (в двух томах): Учебник для студентов университетов и втузов. — М.: Высш, школа, 1981, т. I. — 687 с.
2. Волков Е. А. Численные методы: учебное пособие. – М., Наука, 1982, 254с
3. Самарский А.А. Введение в численные методы. Учебное пособие для вузов – М., Наука, 1987, 288 с.,
4. Шипачёв В. С. Высшая математика. Учеб. для вузов. – М.: Высш. Школа. 1998 – 479 с.
5. Березин И. С., Жидков Н. П. Методы вычислений: учебное пособие. – М.: Физматгиз, 1966.
6. Бахвалов Н. С., Корнев А. А., Чижонков Е. В. Численные методы. Решение задач и упражнения: Учебное пособие – М.: Бином, 2016. – 352с.
7. Иванова Г.С. Основы программирования: Учебник для вузов.-М.: Изд-во МГТУ им. Н. Э. Баумана, 2001
8. Квасов, Б. И. Численные методы анализа и линейной алгебры. Использование Matlab и Scilab: Учебное пособие / Б.И. Квасов. - СПб.: Лань, 2016. - 328 с.
9. Бураков М. В. Основы работы в MATLAB: учебное пособие. – ГУАП, СПб., 2006.

Приложение А Листинги программы

Приложение А.1 Файл «eto_1_razdel»

```
clear;
clc;
%Вводим данные
[f] = Test_cur();
%График
[f_x_vals, f_y_vals] = graf(f);
[x0] = x0_nach;
[e] = e_tochn;
% Итерационный процесс метода Ньютона
nyton(f, x0, e, f_x_vals, f_y_vals);

function [f] = Test_cur()
clear;
clc;
while true
    f_str = input('Введите функцию: ','s');
    f_str = strcat('@(x)', f_str);
    if isempty(f_str)
        disp("Функция не может быть пустой.");
        continue
    end
    % Проверка введенной функции на корректность
    if contains(f_str, ';') || contains(f_str, 'eval') || contains(f_str,
'feval') || contains(f_str, 'for') || contains(f_str, 'while')
        disp("Запрещенные символы или функции в строке функции.");
        continue
    end
    if contains(f_str, '^') && ~contains(f_str, '.^')
        disp("Используйте оператор '.^' для возведения в степень с
плавающей точкой.");
        continue
    end
    if contains(f_str, '*') && ~contains(f_str, '.*')
        disp("Используйте оператор '.*' для умножения.");
        continue
    end
    if contains(f_str, '/') && ~contains(f_str, './')
        disp("Используйте оператор './' для деления.");
        continue
    end
    try
        % Проверка, что функция начинается с '@(x)'
```

```
        if ~startsWith(f_str, '@(x)')
            error('Функция должна начинаться с "@(x)".');
        end
        % Выполнение функции для проверки, что она корректно определена
        f = str2func(f_str);
        x_test = f(0);
        break % прерываем цикл, если функция корректно определена
    catch ME
        disp(ME.message);
        continue % повторяем цикл, если функция некорректно определена
    end
end
end

function [x_vals, y_vals] = graf(f)
while true
    x_min_str = input('Введите левое значение оси x: ', 's');
    x_min_str = strrep(x_min_str, ',', '.');
    x_min_str = str2double(x_min_str);
    if isnan(x_min_str)
        disp("Левая граница должна быть числом, в числе должен быть один
разделитель.");
        continue
    end
    x_max_str = input('Введите правое значение оси x: ','s');
    x_max_str = strrep(x_max_str, ',', '.');
    x_max_str = str2double(x_max_str);
    if isnan(x_max_str)
        disp("Правая граница должна быть числом, в числе должен быть один
разделитель.");
        continue
    end
    if (x_min_str > x_max_str || x_max_str < x_min_str || x_min_str ==
x_max_str)
        fprintf("Левая граница не может быть больше правого и наоборот, а
также не должны быть равны.\n");
        continue
    end
    break
end
x_vals = x_min_str : 0.1 : x_max_str;
y_vals = f(x_vals);
plot(x_vals, y_vals, 'b', 'LineWidth', 2);
hold on;
line([x_min_str, x_max_str], [0, 0], 'Color', 'k', 'LineWidth', 1); % ось
x
line([0, 0], [min(y_vals), max(y_vals)], 'Color', 'k', 'LineWidth', 1); %
ось y
xlim([x_min_str, x_max_str]);
ylim([min(y_vals), max(y_vals)]);
```

```
axis('tight');
xlabel('Ось X');
ylabel('Ось Y');
grid on;
drawnow;
output_args = {x_vals, y_vals};
end

function [x0] = x0_nach()
x0_str = input("Введите начальное приближенное значение x0, используя
точку для разделения дробной части: ", 's');
while true
    x0_str = strrep(x0_str, ',', '.');
    x0 = str2double(x0_str);
    if isnan(x0)
        disp("x0 должно быть числом, в числе должен быть один
разделитель.");
        x0_str = input("Введите значение x0: ", 's');
        continue
    end

    break
end
end

function [e] = e_tochn()
while true
    e_str = input("Введите точность вычисления корня e, используя точку
для разделения дробной части: ", 's');
    e_str = strrep(e_str, ',', '.');
    e = str2double(e_str);
    if isnan(e)
        disp("e должно быть числом, в числе должен быть один
разделитель.");
        continue
    end
    if e <= 0
        fprintf('Точность должна быть больше нуля, использующая точку в
качестве разделителя дробной части\n');
        continue
    end
    break
end
end

function [dfx,x] = nyton(f, x0, e, f_x_vals, f_y_vals)
max_iter = 100;
    for i = 1:max_iter
        fx = f(x0);
```

```
        dfx = (f(x0+e) - f(x0-e)) / (2*e); % вычисляем производную через
определение
        x = x0 - fx/dfx;
        casatelna(f(x0),x0, f_x_vals, f_y_vals, dfx)
        tochka(x0, f);
        % Проверка критерия окончания итерационного процесса
        if abs(x - x0) < e
            fprintf('Уравнение решено за %d итераций\n', i);
            fprintf('Корень уравнения: %.6f\n', x0);
            % Используем встроенную функцию fzero для получения решения
уравнения с точностью e
            x_fzero = fzero(f, x0);
            fprintf('Корень уравнения по встроенной функции fzero:
%.6f\n', x_fzero);
            break;
        elseif i == max_iter
            close all
            fprintf('Уравнение не решено за %d итераций\n', max_iter);
            break;
        end
        x0 = x;
    end
end
```

Приложение А.2 Файл «eto_2_razdel»

```
clc;
clear;
% Ввод значений массива x и y
[x, y] = get_input_points();
% Ввод значения xx
xx = get_xx_input(x);
% Вычисление значения полинома для всех заданных точек
yy = evaluate_polynomial(x,y, xx);
print_results(x, y, xx, yy);
% Построение графика
plot_polynomial(x, y, xx, yy);

function [x, y] = get_input_points()
% Ввод значений массива x и y
x_str = strrep(input("Введите значения массива x, разделяя их пробелами:
",'s'), ',', '.');
while true
    x_cell = strsplit(x_str, ' ');
    x = str2double(x_cell);
    if any(isnan(x)) || length(x) < 2 || length(unique(x)) ~= length(x)
        disp("Массив x должен содержать не менее двух уникальных чисел,
разделенных пробелами и не буквы.");
```

```
        x_str = strrep(input("Введите значения массива x: ", 's'), ',',
'.');
        continue
    end
    [~, idx] = sort(x); % сортируем массив x и получаем индексы
сортировки
    x = x(idx); % переставляем элементы массива x в соответствии с
порядком сортировки
    break
end

y_str = strrep(input("Введите значения массива y, разделяя их пробелами:
",'s'), ',', '.');
while true
    y_cell = strsplit(y_str, ' ');
    y = str2double(y_cell);
    if any(isnan(y)) || length(y) ~= length(x)
        disp("Массив y должен содержать столько же чисел, сколько и
массив x, разделенных пробелами и не буквы.");
        y_str = strrep(input("Введите значения массива y: ", 's'), ',',
'.');
        continue
    end
    y = y(idx); % переставляем элементы массива y в соответствии с
порядком сортировки массива x
    break
end
end


function xx = get_xx_input(x)
% Ввод значения xx
xx_str = strrep(input("Введите значения xx, разделяя их пробелами или
запятыми и используя точку для разделения дробной части: ", 's'), ',',
'.');
while true
    xx_cell = strsplit(xx_str, {' ', ','});
    xx = str2double(xx_cell);
    if any(isnan(xx))
        disp("xx должно быть числом, в числе должен быть один
разделитель.");
        xx_str = strrep(input("Введите значения xx: ", 's'), ',', '.');
        continue
    end
    if any(ismember(xx, x))
        disp("Введите значения xx, которые не совпадают с значениями в
массиве x.");
        xx_str = strrep(input("Введите значения xx: ", 's'), ',', '.');
        continue
    end
    break
```

```
end
end

function yy = evaluate_polynomial(x,y, xx)
% Вычисление значения полинома для всех заданных точек

% Вычисляем разделенные разности
F = calculate_divided_differences(x, y);

yy = zeros(size(xx));
n = length(x);
for k = 1:length(xx)
    xk = xx(k);
    Pk = F(1,1);
    for j=2:n
        prod = 1;
        for i=1:j-1
            prod = prod * (xk - x(i));
        end
        Pk = Pk + F(1,j)*prod;
    end
    yy(k) = Pk;
end
end

function F = calculate_divided_differences(x, y)
% Вычисляем разделенные разности
n = length(x);
F = y'; % инициализация матрицы разделенных разностей
for j = 2:n
    for i = 1:n-j+1
        F(i,j) = (F(i+1,j-1) - F(i,j-1))/(x(i+j-1) - x(i));
    end
end
end

function print_results(x, y, xx, yy)
clc;
% Вывод результата
% Используем polyfit для вычисления коэффициентов интерполяционного
полинома
p = polyfit(x, y, length(x)-1);
% Используем polyval для вычисления значений интерполяционного полинома в
точках xx
pp = polyval(p, xx);
fprintf("Таблица значений x и y:\n");
fprintf("  x         y   \n");
for i = 1:length(x)
    fprintf("%.6f   %.6f\n", x(i), y(i));
```

```
end
fprintf("\n");
fprintf("Значения полинома в точках:\n");
fprintf("  xx         yy        yy(Matlab)  \n");
for i = 1:length(xx)
    fprintf("%.6f   %.6f   %.6f\n", xx(i), yy(i),pp(i));
end
fprintf("\n");
end

function plot_polynomial(x, y, xx, yy)
% Строим график полинома, проходящего через заданные точки и точки,
добавленные пользователем

F = calculate_divided_differences(x, y);

[x_range, y_range] = evaluate_polynomial_range(F, x, xx);
hold on;
plot(x,y,'o','color','b','DisplayName','Набор точек');
plot(xx,yy,'o','color','r','DisplayName','Точки интерполяции');
xlabel('x');
ylabel('y');
title('Полином');
xlim([min(x_range), max(x_range)]);
ylim([min(y_range), max(y_range)]);
axis tight;
box on;
legend();

% Вычисляем новые разделенные разности
x_new = [x xx];
y_new = [y yy];
F_new = calculate_new_divided_differences(x_new, y_new);

% Интерполируем полином на интервале от минимального значения x до
максимального значения xx
[x_range_new, y_range_new] = evaluate_new_polynomial_range(F_new, x_new);

% Обновляем график
plot(x_range_new,y_range_new,'color','k','DisplayName','Полином');
legend();
% Добавление прямых осей x и y
ax = gca;
ax.XAxisLocation = 'origin';
ax.YAxisLocation = 'origin';
ax.Box = 'off';
end

function [x_range, y_range] = evaluate_polynomial_range(F, x, xx)
```

```
% Строим график полинома, проходящего через заданные точки и точки,
добавленные пользователем
x_range = linspace(min([x xx]), max([x xx]), 10000);
y_range = zeros(size(x_range));
n = length(x);
for i=1:length(x_range)
    P = F(1,1);
    for j=2:n
        prod = 1;
        for k=1:j-1
            prod = prod * (x_range(i) - x(k));
        end
        P = P + F(1,j)*prod;
    end
    y_range(i) = P;
end
end

function F_new = calculate_new_divided_differences(x_new, y_new)
% Вычисляем новые разделенные разности
n_new = length(x_new);
F_new = zeros(n_new);
F_new(:,1) = y_new'; % инициализация матрицы разделенных разностей
for j = 2:n_new
    for i = 1:n_new-j+1
        F_new(i,j) = (F_new(i+1,j-1) - F_new(i,j-1))/(x_new(i+j-1) -
x_new(i));
    end
end
end

function [x_range_new, y_range_new] =
evaluate_new_polynomial_range(F_new, x_new)
% Интерполируем полином на интервале от минимального значения x до
максимального значения xx
x_range_new = linspace(min(x_new), max(x_new), 100);
y_range_new = zeros(size(x_range_new));
n_new = length(x_new);
for i=1:length(x_range_new)
    P = F_new(1,1);
    for j=2:n_new
        prod = 1;
        for k=1:j-1
            prod = prod * (x_range_new(i) - x_new(k));
        end
        P = P + F_new(1,j)*prod;
    end
    y_range_new(i) = P;
end
end
```

Приложение А.3 Файл «eto_3_razdel»

```
clc;
clear;
% Ввод значений
[f,A,B,tol] = vvod();
% Вычисление интеграла методом Симпсона с заданной точностью
[s_new_pos] = sympson_kavo(f,A,B,tol);
% Вычисление с помощью встроенных функции matlab
[ss_pos] = vstroina(f,A,B);
% Вывод результатов
vivod(ss_pos, s_new_pos);
grafik_hi(A, B, f);
function [f,A,B,tol] = vvod()
% Ввод функции пользователем
while true
    f_str = input('Введите функцию: ','s');
    f_str = strcat('@(x)', f_str);
    if isempty(f_str)
        disp("Функция не может быть пустой.");
        continue
    end
    % Проверка введенной функции на корректность
    if contains(f_str, ';') || contains(f_str, 'eval') || contains(f_str,
'feval') || contains(f_str, 'for') || contains(f_str, 'while')
        disp("Запрещенные символы или функции в строке функции.");
        continue
    end
    if contains(f_str, '^') && ~contains(f_str, '.^')
        disp("Используйте оператор '.^' для возведения в степень с
плавающей точкой.");
        continue
    end
    if contains(f_str, '*') && ~contains(f_str, '.*')
        disp("Используйте оператор '.*' для умножения.");
        continue
    end
    if contains(f_str, '/') && ~contains(f_str, './')
        disp("Используйте оператор './' для деления.");
        continue
    end
    try
        % Проверка, что функция начинается с '@(x)'
        if ~startsWith(f_str, '@(x)')
            error('Функция должна начинаться с "@(x)".');
        end
        % Выполнение функции для проверки, что она корректно определена
        f = str2func(f_str);
        x_test = f(0);
```

```
        break % прерываем цикл, если функция корректно определена
    catch ME
        disp(ME.message);
        continue % повторяем цикл, если функция некорректно определена
    end
end
% Ввод параметров
A_str = strrep(input("Введите нижнюю часть интеграла: ", 's'), ',', '.');
while true
    A = str2double(A_str);
    if isnan(A)
        disp("Должно быть числом, в числе должен быть один
разделитель.");
        A_str = strrep(input("Введите нижнюю часть интеграла: ", 's'),
',', '.');
        continue
    end
    break
end
B_str = strrep(input("Введите верхнюю часть интеграла: ", 's'), ',',
'.');
while true
    B = str2double(B_str);
    if isnan(B)
        disp("Должно быть числом, в числе должен быть один
разделитель.");
        B_str = strrep(input("Введите верхнюю часть интеграла: ", 's'),
',', '.');
        continue
    end
    break
end
tol_str = strrep(input("Введите точность: ", 's'), ',', '.');
while true
    tol = str2double(tol_str);
    if isnan(tol)
        disp("Должно быть числом, в числе должен быть один
разделитель.");
        tol_str = strrep(input("Введите точность: ", 's'), ',', '.');
        continue
    end
    break
end
end
function [s_new_pos] = sympson_kavo(f,A,B,tol)
% Вычисление интеграла методом Симпсона с заданной точностью
n = 2;
s_old = 0;
s_new_pos = (B-A)*(f(A) + 4*f((A+B)/2) + f(B))/6;
while abs((s_new_pos - s_old)/s_new_pos) > tol && n <= 100
```

```
    s_old = s_new_pos;
    n = n * 2;
    dx = (B-A)/n;
    s_new_pos = 0;
    for i = 0:2:n-2
        x1 = A + dx*i;
        x2 = A + dx*(i+2);
        s_new_pos = s_new_pos + (dx/3)*(f(x1)+4*f(x1+dx)+f(x2));
    end
end
end
function [ss_pos] = vstroina(f,A,B)
% Вычисление интеграла встроенной функцией integral для проверки
ss_pos = integral(f,A,B);
end
function vivod(ss_pos, s_new_pos)
fprintf('Интеграл пользовательским методом = %.3f\n', s_new_pos);
fprintf('Интеграл встроенной функцией integral = %.3f\n', ss_pos);
cerror = ss_pos - s_new_pos;
k = -floor(log10(abs(cerror)));
cerror = floor(cerror*(10^k))/10^k;
fprintf('Погрешность интеграла = %.3f\n', cerror);
end
function grafik_hi(A, B, f)

% Вывод графика функции
xx = linspace(A,B,1000)';
yy = f(xx);
plot(xx,yy,'b');
hold on;
% Построение трапеции по границам интеграла
x_trapezoid = [A B B A];
y_trapezoid = [f(A) f(B) 0 0];
line(x_trapezoid, y_trapezoid, 'Color', 'black');

% Закрашивание площади трапеции
fill(x_trapezoid, y_trapezoid, 'blue', 'FaceAlpha', 0.2, 'LineStyle', '-
')
% Изменение подинтегральной функции на отрезках
n_parts = 70; % количество отрезков параболами
x_part = linspace(A,B,n_parts+1); % задание точек, в которых будут
вершины парабол
y_part = f(x_part); % значения функции подинтегральной функции в вершинах
парабол

% Разбиение на отрезки и построение кривых Безье
for i = 1:n_parts
    % Вычисление координат точек кривой Безье
    x0 = x_part(i);
    x1 = (x_part(i) + x_part(i+1))/2;
```

```
    x2 = x_part(i+1);
    y0 = y_part(i);
    y2 = y_part(i+1);
    y1 = y0 + 2/3*(y_part(i+1) - y_part(i))*(x1 - x0)/(x2 - x0); %
Определение координаты y на точке x1

    % Построение кривой Безье
    t = linspace(0,1,100);
    x = (1-t).^2*x0 + 2*(1-t).*t*x1 + t.^2*x2;
    y = (1-t).^2*y0 + 2*(1-t).*t*y1 + t.^2*y2;

    % Отображение кривой Безье
    plot(x,y,'r');
end

% Настройка графика
title('Приближенная функция методом Симпсона');
xlabel('x');
ylabel('f(x)')
legend('Функция','Границы интеграла','Закрашенная трапеция интеграла под
функцией','Параболы соединенные кривыми Безье')
grid on;
% Добавление прямых осей x и y
ax = gca;
ax.XAxisLocation = 'origin';
ax.YAxisLocation = 'origin';
ax.Box = 'off';

end
```

Приложение В Полный код

```
clear;
clc;
    disp('Главное меню');
    disp('Эта программа разработана для решения трёх численных методов');
    disp('Выберите пункт меню');
    disp('1. Решение нелинейного уравнения методом
Ньютона(касательные)');
    disp('2. Интерполирование функции по формулам Ньютона');
    disp('3. Численное интегрирование методом Симпсона');
    disp('4. Информация об авторе');
    disp('5. Выход');
while true
    x_str = input('Выберите номер пункта программы ','s');
    x_str = str2double(x_str);
    if isnan(x_str)
        disp("Некоректно введен пункт");
```

```
        continue
    end
    if x_str < 1 || x_str > 5
        fprintf('Некоректно введен пункт.\n');
        continue
    end
    break
end
if x_str == 1
    run ('eto_1_razdel.m');
    return;
end
if x_str == 2
    run ('eto_2_razdel.m');
    return;
end
if x_str == 3
    run ('eto_3_razdel.m');
    return;
end
if x_str == 4
    run ('infa_obo_mne.m');
    return;
end
if x_str == 5
clc; clear; close;
disp('Работа программы завершена');
end


%infa_obo_mne файл
clc;
clear;
fprintf("Автор: Панченко Андрей.\n");
fprintf("Делал долго, но получилось.\n");
fprintf("Номер телефона и почту не дам ^_^.\n");
```

Приложение С Основные функции решения программы

Приложение С.1 Файл «nyton»

```
function [dfx,x] = nyton(f, x0, e, f_x_vals, f_y_vals)
max_iter = 100;
    for i = 1:max_iter
        fx = f(x0);
        dfx = (f(x0+e) - f(x0-e)) / (2*e); % вычисляем производную через
определение
        x = x0 - fx/dfx;
        casatelna(f(x0),x0, f_x_vals, f_y_vals, dfx)
        tochka(x0, f);
```

```
        % Проверка критерия окончания итерационного процесса
        if abs(x - x0) < e
            fprintf('Уравнение решено за %d итераций\n', i);
            fprintf('Корень уравнения: %.6f\n', x0);
            % Используем встроенную функцию fzero для получения решения
уравнения с точностью e
            x_fzero = fzero(f, x0);
            fprintf('Корень уравнения по встроенной функции fzero:
%.6f\n', x_fzero);
            break;
        elseif i == max_iter
            close all
            fprintf('Уравнение не решено за %d итераций\n', max_iter);
            break;
        end
        x0 = x;
    end
end
```

Приложение С.2 Файл «evaluate_polynomial»

```
function yy = evaluate_polynomial(x,y, xx)
% Вычисление значения полинома для всех заданных точек

% Вычисляем разделенные разности
F = calculate_divided_differences(x, y);

yy = zeros(size(xx));
n = length(x);
for k = 1:length(xx)
    xk = xx(k);
    Pk = F(1,1);
    for j=2:n
        prod = 1;
        for i=1:j-1
            prod = prod * (xk - x(i));
        end
        Pk = Pk + F(1,j)*prod;
    end
    yy(k) = Pk;
end
end
function F = calculate_divided_differences(x, y)
% Вычисляем разделенные разности
n = length(x);
F = y'; % инициализация матрицы разделенных разностей
for j = 2:n
    for i = 1:n-j+1
```

```
        F(i,j) = (F(i+1,j-1) - F(i,j-1))/(x(i+j-1) - x(i));
    end
end
end
```

Приложение С.3 Файл «sympson_kavo»

```
function [s_new_pos] = sympson_kavo(f,A,B,tol)
% Вычисление интеграла методом Симпсона с заданной точностью
n = 2;
s_old = 0;
s_new_pos = (B-A)*(f(A) + 4*f((A+B)/2) + f(B))/6;
while abs((s_new_pos - s_old)/s_new_pos) > tol && n <= 100
    s_old = s_new_pos;
    n = n * 2;
    dx = (B-A)/n;
    s_new_pos = 0;
    for i = 0:2:n-2
        x1 = A + dx*i;
        x2 = A + dx*(i+2);
        s_new_pos = s_new_pos + (dx/3)*(f(x1)+4*f(x1+dx)+f(x2));
    end
end
end
```